A Primer on Dependency in Polynomial Product: Identify, Exploit, and Trim

Yijian Liu[1,2], Jiangxia Ge[3], Yu Zhang[1,2], Jiabo Wang[1,2], and Xianhui Lu[1,2](✉)

[1] State Key Laboratory of Cyberspace Security Defense, Institute of Information Engineering, Chinese Academy of Sciences, Beijing, China
[2] School of Cyber Security, University of Chinese Academy of Sciences, Beijing, China
[3] China Telecom Quantum Group Ltd., Hefei, China
{liuyijian, luxianhui}@iie.ac.cn

Abstract. Many lattice-based encryption schemes allow a negligible but nonzero decryption failure rate (DFR), which is closely tied to both correctness and security through failure-based attacks. Several module-lattice constructions (e.g., LAC (NIST PQC Round-2), DAWN (ASIACRYPT 2025), average-case noise analysis in FHE) estimate DFR from one-coordinate marginals together with an independence approximation across the coefficients of polynomial products, namely, that treats the noise as uniformly distributed on a sphere. In the rare-event regime relevant to concrete security, this approximation can be optimistic because polynomial convolution introduces structured dependencies with no analogue in unstructured lattice settings. Geometrically, the noise spreads towards a cube rather than a sphere due to the inherent dependencies.

To make this effect explicit, we study polynomial products in the power-of-two cyclotomic ring through a norm-wise decomposition. The decomposition separates an *outer* term (corresponding to the radius of the sphere), which is effectively captured by coefficient-wise models, and an *inner* term (representing the uneven parts of the spherical surface), which is shown as a diagonal energy term and accounts for the convolution-induced dependencies. This gives an exact explanation for the heavier tails observed in polynomial products and for the resulting gap between independence-based estimates and actual failure behavior.

This perspective has consequences for both attacks and design. On the attack side, it gives a principled proxy criterion for constructing high-DFR candidate ciphertexts in failure-based attacks. In particular, it explains how the attack of Guo *et al.* (ASIACRYPT 2019) can target LAC even when the Hamming weights are fixed, and it improves failure-finding efficiency by identifying the underlying class of bad randomness pairs beyond pattern-based subsets. On the design side, it motivates trimming high-dependency samples during key generation and encryption. We first give a certified trimmed DFR bound based on conditional spectral control, then isolate a separate labeled three-vector heuristic for calibrated interpretation, and finally validate both layers on exact-support and moderate-dimension experiments. We formalize the resulting approach as the generic frameworks TrimPKE and TrimKEM, prove security in the QROM while accounting for rejection, and instantiate the framework for LAC and DAWN as case studies.

Keywords: module lattice · decryption failures · PKE · KEM

1 Introduction

As post-quantum cryptography moves toward standardization, lattice-based constructions have gained increasing prominence due to their efficiency, relatively direct reductions to well-studied hardness assumptions, and applicability across a wide range of cryptographic primitives. In particular, the lattice-based encryption Kyber [6] has been selected in the NIST PQC standardization process. A distinctive feature of many lattice-based encryption schemes, however, is that decryption may fail with small probability. This phenomenon is common to constructions based on Learning with Errors (LWE) and its ring/module variants that decode by rounding and tolerate bounded noise. While one can eliminate decryption failures through sufficiently conservative parameter choices as in NTRU Prime [4] and NTRU HRSS [34], this typically incurs noticeable efficiency and bandwidth costs (e.g., larger public keys and ciphertexts). Consequently, modern KEM designs tune parameters so that the decryption failure rate (DFR) is negligible while retaining performance, as exemplified by Kyber [6] and DAWN [42]. In fully homomorphic encryption (FHE), DFR also plays an analogous role: schemes such as BGV [9], BFV [8,22], and CKKS [12,7] estimate and control failure probabilities to ensure functional correctness of the supported evaluations.

DFR also has direct security implications in the chosen-ciphertext (CCA) model. An adversary with access to a decryption oracle can exploit failures to obtain information about the secret key, making careful parameterization essential for balancing efficiency and security. For example, the NIST PQC setting allows up to 2^{64} adversarially chosen ciphertexts, and security against quantum adversaries typically calls for DFR well below 2^{-128}. Failure-based CCA attacks on code/lattice-based schemes have been extensively studied in [21,28,29,33]. Moreover, the advent of failure boosting [19] and subsequent refinements [20] reduces the dominant attack cost to finding the first failing ciphertext, further increasing the importance of accurate tail estimates. And since the IND-CPA-D security has been proposed in [39], DFR plays a more important role in maintaining the security of FHE schemes; several failure-based attacks on FHE schemes have been studied in [10,11,44].

On the other hand, error correcting code (ECC) is a powerful tool to compress the size in the module-lattice-based encryptions, one typical example is LAC [45] (NIST PQC Round-2). However, underestimating DFR in such schemes leaves them vulnerable to failure-based CCA attacks [29]. Also, average-case noise analysis is common for estimating DFR in FHE schemes, as in [5]. Both the DFR estimation used in LAC [45], DAWN [42], and average-case noise estimation methods used in FHE [5,14] share a common assumption: treating the relevant post-convolution coefficients as independent. Several works provide empirical evidence consistent with this assumption in typical regimes [14,41], but these evaluations largely focus on matching the coarse shape of the probability mass function (PMF), rather than quantifying the statistical distance to the true distribution in the extreme tail. For CKKS, for example, the analysis of [7] reports that the mean and standard deviation of the noise deviate from theoretical

predictions by at most 0.01 bits; such moment-level agreement, however, does not rule out substantial discrepancies in rare-event probabilities. Indeed, [25] observed that independence-based models can induce heavy-tailedness in certain settings and analyzed higher moments (e.g., kurtosis), revealing a non-negligible bias relative to independence-based estimates. Further, [15] demonstrated that convolution-induced dependencies can lead to underestimation of DFR in schemes that combine module-lattice noise with error-correcting codes, such as LAC [45]. That work quantified a gap between independence-based estimates and dependence-aware estimates for LAC. However, its analysis still relied on additional assumptions and did not capture dependency effects under fixed Hamming-weight distributions.

1.1 Our Contributions

Several DFR analyses for module-lattice encryption rely on a coefficient-wise independence assumption for polynomial products, which can lead to systematic underestimation in the DFR. We study the dependencies induced by polynomial product and develop a norm-wise framework that both exploits and mitigates their impact. Our goal is to identify where these dependencies matter, exploit them to shape a better failure-based attack, and quantify how much trimming is required to suppress the resulting tail amplification in ECC-relevant settings. Our contributions are as follows.

Identify: a norm-wise dependency model. We introduce a norm-wise perspective that makes the dependency contribution explicit and distinguishes module lattices from the unstructured lattice setting in the tail regime. Using a new decomposition-based view, we separate the behavior explained by coefficient-wise models from the additional effect induced by polynomial convolution. This clarifies when and why independence-based estimators become inaccurate for DFR in schemes where failures are driven by many moderately large coefficients.

Exploit: a dependency-guided failure-finding heuristic. On the attack side, our model explains why failure-based CCA attacks can still find useful candidates even when the relevant Hamming weights are fixed, and it recovers prior pattern-based selection rules as a particularly structured example. It suggests a dependency-guided proxy selector for high-risk candidates. In our small-scale proxy experiment, this selector exhibits a stronger tail signal than the prior pattern proxy under the same search budget.

Trim: a generic protection framework. On the design side, we propose a lightweight mitigation based on rejection sampling that filters highly dependent samples during key generation and encryption. We present a trimming-conditioned worst-case DFR bound with spectral control, while the sharper three-vector estimate is kept as a separate heuristic supplement. We formalize

this approach as two generic frameworks, TrimPKE and TrimKEM, prove their security in the QROM while accounting for rejection, and explicitly analyze the resulting concrete-security loss. We use the LAC and DAWN frameworks as case studies to illustrate the trade-offs between DFRs and efficiency.

1.2 Our Techniques

Lattice-based encryption schemes may rely on different assumptions for key generation (e.g., NTRU/LWE-type assumptions), but their encryption and decryption procedures are uniformly governed by LWE or a structured variant (Ring-/Module-LWE, and likewise LWR variants). This common structure admits a generic decryption condition. Concretely, decryption recovers each message bit m_i from a noisy value of the form $\langle \boldsymbol{a}_i, \boldsymbol{b}_i \rangle + \frac{q}{2} m_i$, where $\boldsymbol{a}_i, \boldsymbol{b}_i$ are (public/secret-derived) vectors and $\langle \boldsymbol{a}_i, \boldsymbol{b}_i \rangle$ is the effective noise term. Let $z \in \mathcal{R}_n$ denote the centered pre-decoding error polynomial, let $\Delta > 0$ denote the per-coordinate decision threshold, and let

$$N_\Delta := \#\{i \in \{0, \ldots, n-1\} : |z_{[i]}| > \Delta\}$$

count the threshold exceedances. For schemes without ECC, the failure event is $\|z\|_\infty > \Delta$, which equals $N_\Delta = 0$; with a hard-decision code correcting up to t coefficient errors, the failure event becomes $N_\Delta > t$. Accordingly, bounding the DFR reduces to understanding the joint tail behavior of these inner products rather than only their one-coordinate marginal law.

To obtain compact keys and ciphertexts and to enable fast arithmetic, many modern designs (e.g., Kyber [6], Saber [16], BAT [24], NEV [51], and DAWN [42]) adopt Ring-/Module-LWE (or Ring-/Module-LWR). In these structured settings, vector entries are coefficient embeddings of ring elements. For a fixed component index k, the entries $(\boldsymbol{a}_i)_{[k]}$ and $(\boldsymbol{a}_j)_{[k]}$ are not independent draws: they are rotations of the same underlying polynomial in a quotient ring $\mathbb{Z}[x]/\varphi$, satisfying $(\boldsymbol{a}_i)_{[k]} \equiv x^{j-i} \cdot (\boldsymbol{a}_j)_{[k]}$. As a consequence, the family of inner products appearing across coordinates in decryption corresponds to different coefficients of the *same* polynomial products. This is the algebraic source of the dependencies that are absent in unstructured lattices, yet are often ignored by analyses that model coefficients as independent.

Making the dependency contribution explicit. To isolate this effect, we view the centered decryption noise schematically as

$$z = \sum_i a_i b_i + \sum_j w_j,$$

where a_i, b_i, w_j are ring elements sampled independently according to the scheme. The dependence of interest arises from the products $a_i b_i$: their coefficient vectors are *not* well approximated by independent coordinates. We capture this phenomenon through a norm-wise comparison between polynomial products and their unstructured matrix analogues. Since most practical module-lattice

schemes use power-of-two cyclotomic rings, we focus on $\mathbb{Z}[x]/(x^n+1)$. For $a, b \in \mathbb{Z}[x]/(x^n+1)$ and $\bar{a}(x) = a(x^{-1})$, we use the following exact identity:

$$\|ab\|_2^2 \;=\; \boxed{\|a\|_2^2\,\|b\|_2^2} \;+\; \boxed{2\big\langle (a\bar{a})_{[1:n/2)},\, (b\bar{b})_{[1:n/2)} \big\rangle},$$

where $(a\bar{a})_{[1:n/2)}$ denotes the vector of coefficients of $a\bar{a}$ from degrees 1 through $n/2-1$. The key point is conceptual: coefficient-wise independence-based analyses effectively capture the first (solid-box) term, while missing the second (dashed-box) term is the dependency carrier induced by convolution. Even when the coefficients of a and b have mean zero so that the dashed-box term has expectation 0, it can contribute substantial tail mass to $\|ab\|_2^2$, producing occasional large spikes that are invisible to moment-level checks. This explains the abnormal norm growth observed empirically and provides a structural interpretation of why rare decryption failures can be underestimated under the independence assumption in module-lattice settings. This helps account for severe underestimation in settings where independence heuristics are applied (e.g., module-lattice-based encryption with ECC and average-case noise analyses in FHE).

Better failures finding. The norm-wise perspective yields a concrete notion of "bad" randomness pairs for schemes whose decoding failures are driven by multiple coefficients above threshold. In particular, the previous analysis in [15] tracks only the outer behavior (captured by the solid-box term), which may fail to explain attacks that remain effective under fixed Hamming-weight sampling, as in the LAC setting [29]. Under the identity above, the explanation is straightforward: even when the outer contribution is essentially fixed, the dependency carrier can fluctuate significantly, thereby increasing the relevant tail risk. This viewpoint also clarifies why the patterns exploited in [29] are effective: they correlate with unusually large values of $(a\bar{a})_{[1:n/2)}$ (and thus a large dependency contribution). Consequently, instead of restricting to a specific pattern family, we can target the underlying cause and obtain a broader selection rule for high-risk candidates. Section 4 evaluates this claim with explicit proxy tail statistics; it is an exploitation experiment, not part of the certified DFR bound of Section 5.1.

Mitigation via trimming. Finally, we use the same model to design a countermeasure that reduces dependency-induced tail amplification without changing the original parameter sets. The key observation is that the dependency carrier is reflected in simple statistics of $\|(a\bar{a})_{[1:n/2)}\|_\infty$; thus, we can filter atypical high-dependency samples by rejection sampling. Section 5.1 first turns this idea into a certified worst-case DFR bound for accepted samples and then gives a separate heuristic supplement for finer-grained interpretation and experimental backup. This rejection sampling mechanism is lightweight, preserves the original scheme structure, and admits a clean security analysis: the only effect is an additional rejection probability that must be accounted for in reductions and concrete bounds. In Section 5, we formalize trimming as generic frameworks (TrimPKE and TrimKEM), prove their security in the QROM while explicitly incorporating rejection, and evaluate instantiations for LAC and DAWN, demonstrating practical efficiency-robustness trade-offs in Table 5.

2 Preliminaries

2.1 Notations

We use $\mathcal{R}_n$ to denote the cyclotomic ring $\mathbb{Z}[x]/(x^n+1)$, where n is a power of 2, and we write $\mathcal{R}_{n,q} := \mathcal{R}_n/q\mathcal{R}_n$. We use $\mathbb{Z}_q$ to represent the centered residue set $\{-\frac{q-1}{2}, \ldots, 0, \ldots, \frac{q-1}{2}\}$, where q is an odd prime. We denote by regular font letters (e.g. a, b) the elements of $\mathcal{R}_n$, by bold lower-case letters (e.g. $\boldsymbol{a}, \boldsymbol{b}$) vectors over $\mathbb{Z}$ or $\mathcal{R}_n$, and by bold upper-case letters (e.g. $\mathbf{A}, \mathbf{B}$) matrices whose entries lie in $\mathcal{R}_n$. For a polynomial a, we denote the coefficient of x^i by $a_{[i]}$, and we use $\bar{a}$ to represent the conjugate polynomial $\bar{a}(x) = a(x^{-1})$. Additionally, we denote the vector formed by the coefficients $(a_{[i]}, a_{[i+1]}, \ldots, a_{[j-1]})$ as $a_{[i:j)} \in \mathbb{Z}^{j-i}$. We write $\|a\|_2 := \|a_{[0:n)}\|_2$. We refer to the squared ℓ_2-norm $\|a\|_2^2$ as the *outer distortion*, and to $\|(a\bar{a})_{[1:n/2)}\|_\infty$ as the *inner distortion*. For $r \in \{0, \ldots, n-1\}$, let $\omega_r := \exp\left(\frac{(2r+1)\pi i}{n}\right)$ denote the negacyclic evaluation points. We use the function $\mathsf{Sample}(\mathcal{X})$ to denote sampling from the distribution $\mathcal{X}$. We use the notation $\mathcal{X}^n$ for the distribution of polynomials in $\mathcal{R}_n$ whose coefficients are drawn from $\mathcal{X}$, and $\mathcal{U}(\mathcal{V})$ for the uniform distribution on the set $\mathcal{V}$.

2.2 Lattice-based Encryption

For simplicity, we adopt a generic encryption framework based on Module-LWE (e.g., Kyber [6]) as a case study, without focusing on implementation details, as shown in Figure 1. Similarly, we consider a generic NTRU-based framework (e.g., NTRU-HPS [30]) at a comparable level of abstraction in Figure 2.

Fig. 1: **Module-LWE-based Encryption**

KeyGen	$\mathsf{Encrypt}(pk = (\mathbf{A}, \boldsymbol{b}), m)$
$\mathbf{A} \leftarrow \mathsf{Sample}(\mathcal{U}(\mathcal{R}_{n,q})^{l\times k})$	$\boldsymbol{r} \leftarrow \mathsf{Sample}((\mathcal{X}_r^n)^l)$
$\boldsymbol{s} \leftarrow \mathsf{Sample}((\mathcal{X}_s^n)^k)$	$\boldsymbol{e}_1 \leftarrow \mathsf{Sample}((\mathcal{X}_{e_1}^n)^k)$
$\boldsymbol{e} \leftarrow \mathsf{Sample}((\mathcal{X}_e^n)^l)$	$e_2 \leftarrow \mathsf{Sample}(\mathcal{X}_{e_2}^n)$
$\boldsymbol{b} \leftarrow \boldsymbol{s}\mathbf{A}^\intercal + \boldsymbol{e} \pmod q$	$\boldsymbol{c}_1 \leftarrow \boldsymbol{r}\mathbf{A} + \boldsymbol{e}_1 \pmod q$
$pk \leftarrow (\mathbf{A}, \boldsymbol{b})$	$c_2 \leftarrow \boldsymbol{r}\boldsymbol{b}^\intercal + e_2 + \lfloor \frac{q}{2} \rceil \cdot m \pmod q$
$sk \leftarrow \boldsymbol{s}$	**return** $(\boldsymbol{c}_1, c_2)$
return (pk, sk)	

$\mathsf{Decrypt}(\boldsymbol{c}_1, c_2, sk = \boldsymbol{s})$
$m \leftarrow \lfloor \frac{2}{q} \cdot (c_2 - \boldsymbol{c}_1\boldsymbol{s}^\intercal \bmod q) \rceil \pmod 2$
return m

Fig. 2: **NTRU-based Encryption**

KeyGen	Encrypt$(pk = h, m)$
1 : $f' \leftarrow \mathsf{Sample}(\mathcal{X}_f^n)$	1 : $t \leftarrow \mathsf{Sample}(\mathcal{X}_t^n)$
2 : $g \leftarrow \mathsf{Sample}(\mathcal{X}_g^n)$	2 : $c \leftarrow p \cdot ht + m \pmod q$
3 : $f \leftarrow p \cdot f' + 1$	3 : **return** c
4 : **if** f is not invertible on $\mathcal{R}_n/q\mathcal{R}_n$ go back to step 1	Decrypt$(c, sk = f)$
5 : $h \leftarrow gf^{-1} \pmod q$	1 : $c' \leftarrow cf \pmod q$
6 : $pk \leftarrow h$	2 : $m \leftarrow c' \pmod p$
7 : $sk \leftarrow f$	3 : **return** m
8 : **return** (pk, sk)	

2.3 Hard-Decision Error-Correcting Code

Let $z \in \mathcal{R}_n$ denote the centered decryption-error polynomial before the final decision rule is applied, and let $\Delta > 0$ be the per-coordinate decoding threshold. A coordinate error occurs at index i when $|z_{[i]}| > \Delta$. For schemes without an outer ECC layer, decryption therefore fails exactly when

$$\|z\|_\infty > \Delta.$$

For schemes with a hard-decision code that corrects up to t coefficient errors, we define

$$N_\Delta := \#\{i \in \{0, \ldots, n-1\} : |z_{[i]}| > \Delta\},$$

so the ECC failure event is $\{N_\Delta > t\}$.

In the Module-LWE framework of Figure 1, the centered decryption error is

$$z := \boldsymbol{r}\boldsymbol{e}^\mathsf{T} - \boldsymbol{e}_1\boldsymbol{s}^\mathsf{T} + e_2,$$

and the standard rounding decoder corresponds to $\Delta = \lfloor q/4 \rceil$ after centering. The same notation will be used throughout the paper: z is always the centered pre-decoding error polynomial, Δ is always the per-coordinate threshold, and N_Δ is always the number of threshold exceedances.

3 Identify the Dependencies in Polynomial Convolution

"The closer you look, the less you see."

— Now You See Me

In this section, we review existing methods for estimating decryption failure rates (DFR) in module-lattice-based encryption schemes that rely on the independence

assumption and highlight their limitation. We then develop the exact norm-wise identities that isolate the relevant dependency terms, explain why inner-distortion statistics are natural trimming proxies, and use centered rejection-sampling experiments to illustrate how trimming suppresses the associated tail amplification.

3.1 Inaccuracy of Independence Assumption

We first review the standard independence-based DFR estimate and identify the point at which it ceases to describe the true joint law. We use the generic Module-LWE framework of Figure 1, with ring $\mathcal{R}_n$. During decryption, one computes

$$c_2 - \boldsymbol{c}_1\boldsymbol{s}^\mathsf{T} \equiv \boldsymbol{r}\boldsymbol{e}^\mathsf{T} - \boldsymbol{e}_1\boldsymbol{s}^\mathsf{T} + e_2 + \left\lfloor \frac{q}{2} \right\rceil \cdot m \pmod q \tag{1}$$

Let

$$z \triangleq \boldsymbol{r}\boldsymbol{e}^\mathsf{T} - \boldsymbol{e}_1\boldsymbol{s}^\mathsf{T} + e_2$$

be the resulting centered error polynomial. Without an outer ECC layer, the failure event is $\|z\|_\infty > \Delta$ with $\Delta = \lfloor q/4 \rceil$; with a t-error-correcting hard-decision code, the failure event is $N_\Delta > t$ as defined in Section 2.3.

The standard estimation method begins by computing the marginal distribution $\mathcal{X}_z$ of a single coefficient of z. The distribution of e_2 is straightforward, as its coefficients are independent by design. In contrast, the convolution terms $\boldsymbol{r}\boldsymbol{e}^\mathsf{T}$ and $\boldsymbol{e}_1\boldsymbol{s}^\mathsf{T}$ are more involved. In the no-ECC setting, the conventional estimate is the union bound

$$\mathrm{DFR} = \Pr[\|z\|_\infty > \Delta] \le \sum_{i=0}^{n-1} \Pr\big[|z_{[i]}| > \Delta\big] = n \cdot \Pr_{x \leftarrow \mathcal{X}_z}[|x| > \Delta]\,. \tag{2}$$

This bound matches the no-ECC decision rule but does not describe schemes with ECC, where several moderately large coefficients can jointly trigger failure. The standard workaround is to replace the joint law of the indicators N_Δ by an i.i.d. Bernoulli model with parameter $p_\Delta := \Pr_{x \leftarrow \mathcal{X}_z}[|x| > \Delta]$. For example, if the ECC corrects up to t coefficient errors, the resulting binomial model becomes

$$\mathrm{DFR} = 1 - \sum_{i=0}^{t} \binom{n}{i} \left(\Pr_{x \leftarrow \mathcal{X}_z}[|x| > \Delta] \right)^i \left(1 - \Pr_{x \leftarrow \mathcal{X}_z}[|x| > \Delta] \right)^{n-i}, \tag{3}$$

where t is the maximum number of errors the ECC used can correct. This expression is an independence-based approximation, not a theorem about the true DFR. In practice, the marginal law $\mathcal{X}_z$ is often computed numerically (e.g., via scripts [1] used in Kyber [6]). The main limitation is therefore not the one-coordinate marginal, but the replacement of the joint law by independent Bernoulli trials. While the central limit theorem provides asymptotic guidance

[1] https://github.com/pq-crystals/security-estimates/blob/master/Kyber_failure.py

under independence as $n \to \infty$, the target DFRs are typically exponentially small (e.g., below 2^{-128}). In this regime, even weak dependence can materially change the tail probability of N_Δ.

To illustrate the effect of structural dependencies, let $a, b \in \mathcal{R}_n$. Although the pairwise covariance of distinct coefficients in the product ab can vanish on average for centered samplers [41], this does not imply that the joint tail behaves as if the coefficients were independent. We therefore study the squared ℓ_2-norm as a structural diagnostic: it aggregates many moderately large coefficients and reacts to the same kind of coefficient clustering that later enters the ECC event $N_\Delta > t$, even though it is not itself the failure criterion. Specifically, we sampled 2^{32} polynomial pairs $(a, b) \in \mathcal{R}_{256} \times \mathcal{R}_{256}$ with coefficients drawn from $\mathcal{U}(\mathbb{Z}_3)$, and compared the squared ℓ_2-norms of their products against those from 2^{32} matrix-vector products $(\mathbf{A}, \boldsymbol{b}) \in \mathbb{Z}^{256\times256} \times \mathbb{Z}^{256\times1}$, where all entries are independently sampled from $\mathcal{U}(\mathbb{Z}_3)$. The comparison is performed over integer products before reduction modulo q, so the plot isolates convolution-induced dependence rather than wrap-around effects.

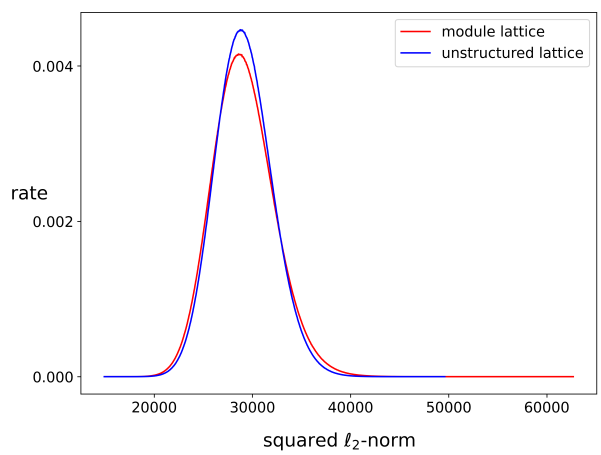


Fig. 3: Empirical distributions of $\|ab\|_2^2$ for integer negacyclic products (red) and $\|\mathbf{A}\boldsymbol{b}\|_2^2$ for an i.i.d. matrix baseline (blue).

Figure 3 highlights a clear discrepancy: polynomial products inherit hidden coefficient-wise dependencies corresponding to the module lattice, while the matrix case represents a genuine i.i.d. baseline corresponding to the unstructured lattice. The x-axis shows the squared ℓ_2-norm, and the y-axis shows the frequency of observing that value. The red markers plot the empirical distribution of $\|ab\|_2^2$, while the blue markers represent $\|\mathbf{A}\boldsymbol{b}\|_2^2$. Thus, the blue curve corresponds to the independence-based prediction commonly assumed in existing analyses, whereas the red curve captures the actual behavior of polynomial products. Notably, the distribution of $\|ab\|_2^2$ exhibits a substantially heavier right tail, meaning that

large-norm events occur far more frequently than the i.i.d. model predicts. This mismatch shows that matching one-coordinate marginals does not control the joint tail relevant to DFR.

3.2 Norm-wise Model

We now record the exact algebraic identities behind the norm-wise model. The main point is that the contribution missed by the coefficient-wise approximation is not a vague geometric effect, but a concrete cross term built from negacyclic autocorrelations.

Lemma 1 (Norm identity). *For every $a \in \mathcal{R}_n$,*

$$\|a\|_2^2 \;=\; (a\bar{a})_{[0]}. \tag{4}$$

Proof. Since $a\bar{a} = \sum_{u,v} a_{[u]} a_{[v]} x^{u-v}$, then we have $(a\bar{a})_{[0]} = \sum_{u=0}^{n-1} a_{[u]}^2 = \|a\|_2^2$.

Lemma 2 (Norm decomposition). *For every $a, b \in \mathcal{R}_n$,*

$$\|ab\|_2^2 \;=\; (ab\bar{a}\bar{b})_{[0]} \;=\; \boxed{\|a\|_2^2\,\|b\|_2^2} \;+\; \boxed{2\big\langle (a\bar{a})_{[1:n/2)},\, (b\bar{b})_{[1:n/2)} \big\rangle}. \tag{5}$$

Proof. By Lemma 1,

$$\|ab\|_2^2 = ((ab)\overline{(ab)})_{[0]} = ((a\bar{a})(b\bar{b}))_{[0]}.$$

Because $\overline{a\bar{a}} = a\bar{a}$, the coefficients of $a\bar{a}$ satisfy $(a\bar{a})_{[n-k]} = -(a\bar{a})_{[k]}$ for $1 \le k < n$ and $(a\bar{a})_{[n/2]} = 0$; hence

$$a\bar{a} = (a\bar{a})_{[0]} + \sum_{k=1}^{n/2-1} (a\bar{a})_{[k]} (x^k - x^{n-k}),$$

and analogously for $b\bar{b}$. Extracting the coefficient of x^0 in the product gives

$$((a\bar{a})(b\bar{b}))_{[0]} = (a\bar{a})_{[0]} (b\bar{b})_{[0]} + 2 \sum_{k=1}^{n/2-1} (a\bar{a})_{[k]} (b\bar{b})_{[k]},$$

which is exactly (5).

Lemma 3 (Coefficient inner-product identity). *For every $a, b \in \mathcal{R}_n$ and every $i \in \{0, \ldots, n-1\}$,*

$$(ab)_{[i]} = \big\langle a_{[0:n)}, (x^i \cdot \bar{b})_{[0:n)} \big\rangle. \tag{6}$$

Consequently, for any i, j,

$$\big\langle (x^i \cdot \bar{b})_{[0:n)}, (x^j \cdot \bar{b})_{[0:n)} \big\rangle = (b\bar{b})_{[j-i]}. \tag{7}$$

Proof. The coefficient $(ab)_{[i]}$ is the negacyclic convolution sum $\sum_{u=0}^{n-1} a_{[u]}(x^i\bar{b})_{[u]}$, which is exactly the inner product in (6). Applying the same identity to the pair $(x^i\bar{b}, \overline{x^j\bar{b}})$ yields (7).

Equation (5) separates the diagonal energy term $\|a\|_2^2\|b\|_2^2$ from the signed cross term $\langle (a\bar{a})_{[1:n/2)}, (b\bar{b})_{[1:n/2)}\rangle$. When the distributions of the coefficients of a and b are both centered at 0, the expectation of the covariance is 0, as shown in [41], meaning the expectation of $\left\langle (a\bar{a})_{[1:n/2)}, (b\bar{b})_{[1:n/2)}\right\rangle$ is zero. However, the fluctuations in covariance still lead to heavy-tailed behavior in $\|ab\|_2^2$, thickening the tail of the distribution and thereby increasing the DFR for schemes that rely on the independence assumption. When asymmetric error distributions are used, the dashed-box term can contribute much larger values, exacerbating tail amplification. Equation (6) also explains why inner distortion governs joint tail events. If the same vector $a_{[0:n)}$ has unusually large inner products with two shifted copies of $\bar{b}$, then those shifts must overlap substantially, and Equation (7) shows that their overlap is measured exactly by $(b\bar{b})_{[j-i]}$. This is the algebraic mechanism by which large off-zero autocorrelations make several coefficients of ab large simultaneously.

Remark 1. For the tri-cyclotomic ring defined by $\mathbb{Z}[x]/(x^n - x^{n/2} + 1)$, with $n = 2^i 3^j$, as used in certain NTRU-based schemes [46,37,40], Lemmas 1 and 2 do not hold in this form. Extending the norm-wise model to that setting requires additional analysis, along with the related analysis in [32].

Heuristic 1 (Asymptotic threshold-selection guide) *To calibrate exploratory trim thresholds, assume that the coefficients of a are modeled by an i.i.d. centered Gaussian surrogate with variance σ^2, and further approximate the nonzero coefficients of $a\bar{a}$ by independent centered Gaussians with variance $n\sigma^4$. Under this surrogate,*

$$\mu_{\text{outer}} := \sigma^2 n,\ \mu_{\text{inner}} := \sigma^2\sqrt{n}\left(\sqrt{2\log\left(\frac{n}{2}-1\right)} - \frac{\log\log\left(\frac{n}{2}-1\right)+\log\pi}{2\sqrt{2\log\left(\frac{n}{2}-1\right)}}\right)$$
$$\sigma_{\text{outer}} := \sigma^2\sqrt{2n},\ \sigma_{\text{inner}} := \frac{\pi\,\sigma^2\sqrt{n}}{\sqrt{12\log\left(\frac{n}{2}-1\right)}} \tag{8}$$

serve as heuristic location and scale parameters for the outer statistic $\|a\|_2^2$ and the inner score $\|(a\bar{a})_{[1:n/2)}\|_\infty$. We use them only for threshold selection in the exploratory experiments below; the certified DFR analysis of Section 5.1 does not rely on this surrogate model.

Motivated by Heuristic 1, we use the centered trimming predicate

$$\begin{cases} \left|\|a\|_2^2 - \mu_{\text{outer}}\right| > \omega_{\text{outer}}\cdot\sigma_{\text{outer}}, \\ \left|\|(a\bar{a})_{[1:n/2)}\|_\infty - \mu_{\text{inner}}\right| > \omega_{\text{inner}}\cdot\sigma_{\text{inner}}, \end{cases} \tag{9}$$

and reject any sample a (resp. b) that violates it. A centered rule is preferable here to one-sided trimming because it perturbs the outer-energy distribution less severely; in fixed-weight samplers such as LAC, the outer term is already nearly fixed, and later sections therefore trim only the inner statistic. To isolate the convolution effect, Figure 4 uses $\omega_{\text{outer}} = +\infty$ together with $\omega_{\text{inner}} \in \{1, 2\}$. As trimming becomes stricter, the polynomial-product distribution moves closer to the unstructured baseline, confirming that the excess tail mass comes from convolution-induced dependence.

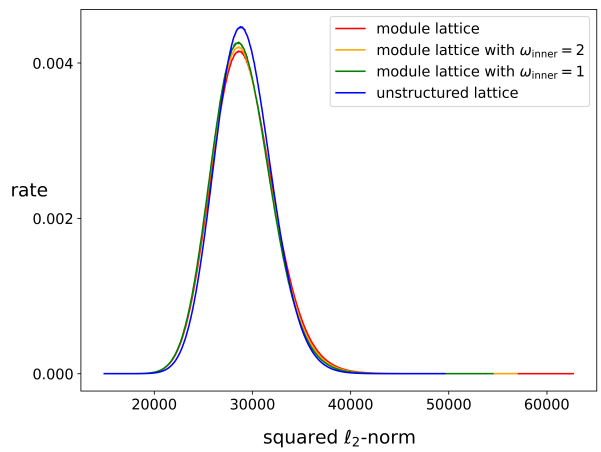


Fig. 4: Empirical effect of centered inner-distortion trimming on $\|ab\|_2^2$. Tightening the trim parameter removes the strongest dependency outliers and contracts the polynomial-product tail toward the i.i.d. matrix baseline.

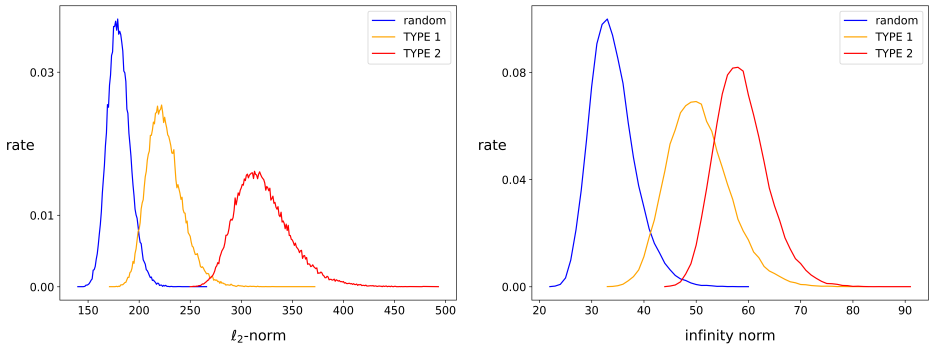
4 Exploit Dependencies in CCA Attacks of Module-lattice-based Encryptions with ECC

This norm-wise perspective also yields an attack-side heuristic for cryptanalysis. A key step in CCA attacks leveraging high DFR is to craft "bad" ciphertexts whose failure probability is substantially above average. In the LAC setting, the attacker searches over candidate fixed-weight randomness polynomials drawn from the same sampler as the scheme and looks for instances whose dependency score predicts elevated tail behavior. Prior work describes this search using message-derived patterns because those patterns induce particular instances of randomness [29]; our point here is more generic: the relevant predictor is the resulting polynomial dependency profile, rather than a specific syntactic pattern family. The effectiveness of patterns in [29] is therefore explained by the large inner distortion they induce. Our norm-wise model makes this dependency

explicit and suggests a dependency-guided proxy selector rather than a fixed-pattern heuristic.

In LAC encryption, the distributions $\mathcal{X}_r$ and $\mathcal{X}_{e_1}$ are fixed Hamming weight distributions, which effectively fix the *outer distortion*. From the norm-wise perspective, the relevant selector is therefore the inner-distortion score $\|(a\bar{a})_{[1:n/2)}\|_\infty$ from Section 3.2. The adversarial strategy of [29] focuses on polynomials with an interval that contains only 1 or -1 and the rest are 0s, which was called the TYPE 2 pattern, and the TYPE 1 pattern is a special case with no 0 in the interval. While such patterns do indeed yield large inner distortion, our model shows that they are merely a special case within a larger class of exploitable ciphertexts; any polynomial whose coefficients exhibit the large *inner distortion* tends to produce a heavier upper tail in the resulting product distribution.

To demonstrate that these patterns actually possess the property of high distortion empirically, we conducted a simple experiment in $\mathcal{R}_{512}$, using the LAC NIST-I sampler: 128 coefficients equal to $+1$, 128 equal to -1, and the remainder equal to 0. We first sampled a baseline pool of 2^{16} random polynomials a from this distribution, a pool of 2^{16} random polynomials b_1 from the TYPE 1 distribution with 37 consecutive 1's, and a pool of 2^{16} random polynomials b_2 from the TYPE 2 distribution with an interval that contains 60 1's (no need to be consecutive) and 42 0s. We note that the probabilities of the appearance of these two patterns are approximate, $2^{-65.41}$ versus $2^{-65.35}$, calculated from the equation in Section 3.1 in [29]. We then calculate the infinity norm and ℓ_2 norm of $(a\bar{a})_{[1:n/2)}$ for each pool, and draw the figures as follows.



(a) comparison of ℓ-2 norm

(b) comparison of infinity norm

Fig. 5: Comparison of inner distortion

We can see that TYPE 2 has larger norms than TYPE 1 than the uniformly sampled one. As noted in [29], TYPE 2 performs better than TYPE 1 at the same probability; this figure directly supports this. The *inner distortion* serves as a metric to evaluate the quality of the pattern, which implies that we could use this metric to find more patterns with more effects on decryption failure.

Moreover, to illustrate the benefits of the *inner distortion* metric check rather than the specific pattern check. We conduct another simple experiment in $\mathcal{R}_{512}$, still using the LAC NIST-I sampler. We first fix 100 random polynomials a from the distribution with fixed hamming weight 256 as the secret keys, and then scan an additional 2^{18} candidates under each of the two selection criteria below:

- *Inner-distortion selector:* retain polynomials $b \in \mathcal{R}_{512}$ satisfying

$$\left\|(b\bar{b})_{[1:n/2)}\right\|_\infty \;\geq\; 40.$$

- *Single-signal interval selector* (TYPE 2): retain polynomials $c \in \mathcal{R}_{512}$ containing an interval with at least 11 positive entries or 11 negative entries, and the rest are at most 19 zeros.

By following the notations in Section 2.3, we calculate N_Δ with the threshold $\Delta = 40$ for each polynomial product ab and ac and treat $N_\Delta \geq 3$ as the ECC failure event. These are empirical tail probes chosen to yield measurable variation within feasible Monte Carlo budgets; they are not the LAC decoding rule.

The cutoffs 40 and $(11, 19)$ are empirical rare-event selectors chosen only for this comparison. After the same search budget of 2^{18} trials, we obtained about $2^{20.78}$ valid samples of the inner-distortion selector and about $2^{20.45}$ valid samples of the continuous-run selector. Summing the proxy statistic N_Δ over the retained samples gives 64 exceedances for the ab family (inner-distortion selector) and 43 for the ac family (continuous-run selector). Thus, under the same search budget, the inner-distortion selector produces 18% more ECC failure probability. We view this as evidence that the norm-wise score is a useful failure-finding heuristic.

We also use the same fixed-weight sampler to test the trimming rule from the defensive direction. Specifically, we retain only those randomness pairs (a, b) satisfying $\|(a\bar{a})_{[1:n/2)}\|_\infty \leq 30$ and $\|(b\bar{b})_{[1:n/2)}\|_\infty \leq 30$. In this experiment, the cluster proxy $\sum C_{40,5}(ab)$ drops from 24 to 1 after trimming. This is again a proxy statistic rather than the full ECC failure probability: the cutoff 40 and the five-coordinate cluster rule are used only to compare tail concentration under matched methodology. The experiment, therefore, supports a narrower claim than a DFR estimate: namely, that trimming suppresses the high-dependency samples that dominate this attack-oriented proxy.

5 Trim on Module Lattice Encryptions and Encapsulations

Given the weaknesses introduced by dependencies, in this section we present a generic, lightweight safeguard procedure for module-lattice-based public key encryption (PKE) and key encapsulation mechanisms (KEM). The central idea is to reject polynomials whose dependence statistics would otherwise lead to more severe tail behavior. Outer-distortion screening is already familiar in lattice cryptography, for example, in Falcon key generation [23] and BAT-style rejection [24], where public predicates filter large-norm outliers. We separate that familiar outer

screening from the dependencies statistics identified. We bridge the dependency to DFR via the canonical embedding, and then introduce a rigorous upper bound of DFR based on the bounded spectrum and a heuristic but tight upper bound of DFR based on the bounded *inner distortion*. Furthermore, given a module-lattice-based PKE scheme with algorithms KeyGen, Encrypt, and Decrypt, we first introduce the generic protection framework TrimPKE, then analyze its impact on concrete security, and finally describe techniques for efficient implementation. By applying the Fujisaki-Okamoto transform to TrimPKE, we obtain the corresponding TrimKEM. The full security proofs of TrimPKE and TrimKEM are provided in Section 6. The framework keeps the arithmetic parameters unchanged, but it introduces explicit rejection rates, runtime overhead, and support-loss terms that must be accounted for in the concrete security analysis. We instantiate it in LAC [45] and DAWN [42] as case studies.

5.1 From Dependency to DFR

In order to transfer the quantifiable dependency to the analysis of DFR, we show the viewpoint of *inner distortion* after canonical embedding.

Lemma 4 (Canonical embedding identity). *For every $a \in \mathcal{R}_n$ and every $k \in \{0, \ldots, n-1\}$,*

$$(a\bar{a})_{[k]} = \frac{1}{n} \sum_{r=0}^{n-1} |a(\omega_r)|^2 \omega_r^{-k}. \tag{10}$$

Proof. The negacyclic Fourier transform diagonalizes multiplication in $\mathcal{R}_n$, so $(a\bar{a})(\omega_r) = a(\omega_r)\overline{a(\omega_r)} = |a(\omega_r)|^2$. Equation (10) is the inverse Fourier expansion of this power spectrum.

The vector $(a\bar{a})_{[1:n/2)}$ can be read as the collection of off-zero negacyclic autocorrelations, as the nonconstant Fourier modes of the power spectrum $|a(\omega_r)|^2$, as the overlap profile between shifted copies of $\bar{a}$, and as the trimming score used later in the paper. Equation (10) shows that the nonzero coefficients of $a\bar{a}$ are exactly the nonconstant Fourier modes of the power spectrum $r \mapsto |a(\omega_r)|^2$. Thus, a large inner distortion means that the spectral energy of a is anisotropic across the negacyclic embedding coordinates.

5.1.1 Rigorous bound from conditional covariance control

We then propose a dependency-aware DFR analysis for module-lattice-based schemes with hard-decision ECC. By following the notations in Section 2.3, we use the exact decryption-noise model

$$z = \sum_{u=1}^{m} a_u b_u + \eta,$$

where $A := (a_1, \ldots, a_m)$ collects the left factors whose trim statistics are controlled, the polynomials b_u have independent centered coefficients with coefficient

variances σ_u^2, and η is an independent centered additive term with coefficient variance σ_η^2. For the generic module lattice-based encryption, one may take

$$z = \sum_{j=1}^{l} r_j e_j - \sum_{j=1}^{k} e_{1,j} s_j + e_2 \tag{11}$$

and condition on $A = (r_1, \ldots, r_l, -e_{1,1}, \ldots, -e_{1,k})$ by isolating the multiplicative term (e.g. $x - 1$ in NTRU-HRSS [34]) and absorbing the compression/rounding contribution into η. In particular, the variables z, Δ, t, and N_Δ are fixed before any dependence proxy is introduced. This coefficient-i.i.d. setting is the abstract regime in which the theorem below is stated. When a concrete sampler matches these hypotheses, the resulting bound is certified.

Let $\omega_r := \exp\left(\frac{(2r+1)\pi i}{n}\right)$ be the negacyclic evaluation points and let F_n denote the corresponding unitary Fourier matrix. Conditioned on A, each map $b_u \mapsto a_u b_u$ is diagonalized by F_n, and since the coefficients of b_u are independent, centered, and have covariance σ_u^2, the conditional covariance of z takes the form

$$\mathrm{Cov}(z \mid A) = F_n^* \,\mathrm{diag}(\lambda_0(A), \ldots, \lambda_{n-1}(A)) F_n, \qquad \lambda_r(A) = \sigma_\eta^2 + \sum_{u=1}^{m} \sigma_u^2 |a_u(\omega_r)|^2.$$

For sampler classes whose right factors have independent centered coefficients, this template applies directly. For LAC, one can still use Equation (11), but because the fixed-Hamming-weight factors are not coefficient-wise independent, the latter LAC case-study rows are reported only as conditional/empirical diagnostics rather than as direct certified corollaries of the theorem below.

Based on the link with inner distortion and canonical embedding from 10, we define

$$\Lambda(A) := \max_{0 \le r < n} \lambda_r(A), \qquad \Lambda_{\mathrm{trim}} := \sup_{A \in \mathcal{A}_{\mathrm{trim}}} \Lambda(A),$$

where $\mathcal{A}_{\mathrm{trim}}$ is the set of accepted left-factor tuples.

Lemma 5 (Trim-to-spectrum control). *For every $a \in \mathcal{R}_n$,*

$$\max_{0 \le r < n} |a(\omega_r)|^2 \le (a\bar{a})_{[0]} + 2 \sum_{k=1}^{n/2-1} \left|(a\bar{a})_{[k]}\right|.$$

Consequently, if an accepted sample satisfies $\|(a\bar{a})_{[1:n/2)}\|_\infty \le B_{\mathrm{inner}}$, then

$$\max_{0 \le r < n} |a(\omega_r)|^2 \le \|a\|_2^2 + (n-2) B_{\mathrm{inner}}.$$

Proof. By Lemma 4,

$$|a(\omega_r)|^2 = \sum_{k=0}^{n-1} (a\bar{a})_{[k]} \omega_r^k.$$

Taking absolute values and using $|\omega_r| = 1$ gives

$$|a(\omega_r)|^2 \le (a\bar{a})_{[0]} + \sum_{k=1}^{n-1} \big|(a\bar{a})_{[k]}\big|.$$

Because $(a\bar{a})_{[n-k]} = -(a\bar{a})_{[k]}$ for $1 \le k < n/2$ and $(a\bar{a})_{[n/2]} = 0$, the nonzero terms occur in pairs of equal magnitude, so

$$\sum_{k=1}^{n-1} \big|(a\bar{a})_{[k]}\big| = 2 \sum_{k=1}^{n/2-1} \big|(a\bar{a})_{[k]}\big|.$$

This proves the first claim, and the second follows from $(a\bar{a})_{[0]} = \|a\|_2^2$ together with the bound $\big|(a\bar{a})_{[k]}\big| \le B_{\text{inner}}$ for $1 \le k < n/2$.

Thus trimming $\|(a_u\bar{a}_u)_{[1:n/2)}\|_\infty$ suppresses the nonconstant Fourier modes of the power spectrum $r \mapsto |a_u(\omega_r)|^2$ and yields an explicit spectral envelope. For example, if each accepted a_u additionally satisfies $\|(a_u\bar{a}_u)_{[1:n/2)}\|_\infty \le B_u$ and $\|a_u\|_2^2 \le E_u$, then

$$\Lambda(A) \le \sigma_\eta^2 + \sum_{u=1}^{m} \sigma_u^2\big(E_u + (n-2)B_u\big). \tag{12}$$

This is the rigorous bridge from the trimming statistic to the spectral control parameter used in the certified DFR bound.

Theorem 1 (Rigorous DFR upper bound under trimmed spectrum). *Assume that for every accepted $A \in \mathcal{A}_{\text{trim}}$, the conditional random vector $z_{[0:n)}$ is centered and has covariance*

$$\operatorname{Cov}(z \mid A) = F_n^* \operatorname{diag}(\lambda_0(A), \dots, \lambda_{n-1}(A)) F_n.$$

Then the trimmed worst-case DFR satisfies

$$\Pr[N_\Delta > t \mid A \in \mathcal{A}_{\text{trim}}] \le \text{DFR}_{\text{dep,upper}} := \binom{n}{t+1} 2^{t+1} \cdot \frac{\Lambda_{\text{trim}}}{(t+1)\Delta^2}. \tag{13}$$

Proof. Fix an accepted tuple $A \in \mathcal{A}_{\text{trim}}$. If $N_\Delta > t$, then there exists a subset $S \subseteq \{0, \dots, n-1\}$ with $|S| = t+1$ such that $|z_{[i]}| > \Delta$ for every $i \in S$. For this subset choose signs $\varepsilon_i := \operatorname{sgn}(z_{[i]})$. Then

$$\sum_{i \in S} \varepsilon_i z_{[i]} = \sum_{i \in S} |z_{[i]}| > (t+1)\Delta.$$

Hence

$$\{N_\Delta > t\} \subseteq \bigcup_{\substack{S \subseteq \{0,\dots,n-1\} \\ |S| = t+1}} \bigcup_{\varepsilon \in \{\pm 1\}^S} \left\{ \sum_{i \in S} \varepsilon_i z_{[i]} > (t+1)\Delta \right\}.$$

For fixed S and ε, set $x = \sum_{i \in S} \varepsilon_i e_i$, where e_i denotes the i-th standard basis vector of $\mathbb{R}^n$. Then $\|x\|_2^2 = t+1$, and

$$Y_{S,\varepsilon} := \sum_{i \in S} \varepsilon_i z_{[i]} = \langle x, z_{[0:n)} \rangle$$

is centered conditional on A. Moreover,

$$\mathrm{Var}(Y_{S,\varepsilon} \mid A) = x^\top \mathrm{Cov}(z \mid A) x \le \Lambda(A) \|x\|_2^2 = \Lambda(A)(t+1) \le \Lambda_{\mathrm{trim}}(t+1).$$

Therefore, by Chebyshev's inequality,

$$\Pr[Y_{S,\varepsilon} > (t+1)\Delta \,|\, A] \le \Pr[|Y_{S,\varepsilon}| > (t+1)\Delta \,|\, A] \le \frac{\mathrm{Var}(Y_{S,\varepsilon} \mid A)}{(t+1)^2 \Delta^2} \le \frac{\Lambda_{\mathrm{trim}}}{(t+1)\Delta^2}.$$

There are $\binom{n}{t+1}$ choices of S and 2^{t+1} sign patterns, so the union bound gives

$$\Pr[N_\Delta > t \mid A] \le \binom{n}{t+1} 2^{t+1} \cdot \frac{\Lambda_{\mathrm{trim}}}{(t+1)\Delta^2}.$$

Since the right-hand side is uniform over all accepted A, the same bound holds after conditioning only on the trim event $A \in \mathcal{A}_{\mathrm{trim}}$.

Remark 2. Equation (13) is fully rigorous under conditional covariance control alone, but it is conservative and may be numerically loose. Its role is to provide a certified safety envelope showing that trimming suppresses the relevant spectral variance parameter. Sharper practical predictions require additional concentration assumptions or separate heuristic models.

5.1.2 Heuristic three-vector supplement

To obtain a sharper bound, we now present the three-vector model (visualized in Figure 6) as a heuristic single-coordinate approximation. It is used for calibrated interpretation.

Lemma 6 (Exact projection lemma). *Let $a, b \in \mathcal{R}_n$ and fix $i \neq 0$. Define*

$$S_0 := (ab)_{[0]}, \qquad C_i := (b\bar{b})_{[i]}, \qquad R_a := \|a\|_2, \qquad R_b := \|b\|_2,$$

and

$$v_1 := a_{[0:n)}, \qquad v_2 := \bar{b}_{[0:n)}, \qquad v_3 := (x^i \bar{b})_{[0:n)}.$$

Then

$$\langle v_1, v_2 \rangle = S_0, \qquad \langle v_1, v_3 \rangle = (ab)_{[i]}, \qquad \langle v_2, v_3 \rangle = C_i,$$

and

$$(ab)_{[i]} = \mu_i + \delta_i, \qquad \mu_i := \frac{S_0 C_i}{R_b^2}, \qquad |\delta_i| \le \sqrt{R_a^2 - \frac{S_0^2}{R_b^2}} \sqrt{R_b^2 - \frac{C_i^2}{R_b^2}}.$$

Proof. The three displayed inner-product identities are immediate from Equation (6). Decompose v_1 and v_3 orthogonally with respect to v_2:

$$v_1 = \frac{S_0}{R_b^2} v_2 + w, \qquad v_3 = \frac{C_i}{R_b^2} v_2 + u_i,$$

where $w, u_i \perp v_2$. Then

$$(ab)_{[i]} = \langle v_1, v_3 \rangle = \frac{S_0 C_i}{R_b^2} + \langle w, u_i \rangle.$$

This gives the claimed decomposition with $\delta_i := \langle w, u_i \rangle$. Finally,

$$\|w\|_2^2 = R_a^2 - \frac{S_0^2}{R_b^2}, \qquad \|u_i\|_2^2 = R_b^2 - \frac{C_i^2}{R_b^2},$$

so the bound on $|\delta_i|$ follows from Cauchy–Schwarz.

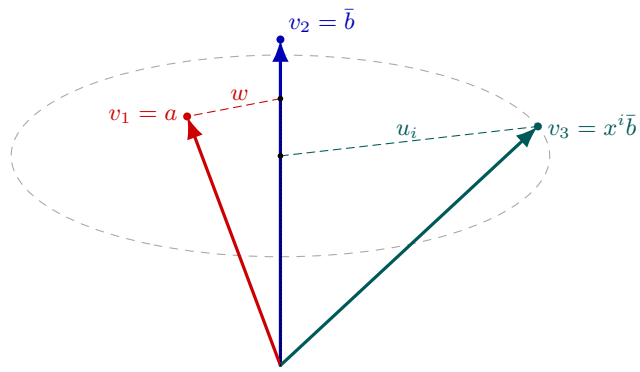


Fig. 6: Geometric visualization of the three-vector model. The axis vector is $v_2 = \bar{b}$, the vector $v_1 = a$ is close to the axis, and the shifted copy $v_3 = x^i \bar{b}$ has the same norm as v_2 and a prescribed axial correlation C_i. The dashed segments indicate the orthogonal components in the decompositions of Theorem 6.

Heuristic 2 (Dependence-aware heuristic DFR estimate) *Fix $i \neq 0$ and condition on b, R_a, S_0, and C_i. In the decomposition of Theorem 6, assume that the normalized vector $w/\|w\|_2$ is approximately uniform on the unit sphere of $v_2^\perp$ and approximately independent of the direction of u_i. Then*

$$(ab)_{[i]} \approx \mu_i + \sigma_i W_i, \qquad \sigma_i := \sqrt{R_a^2 - \frac{S_0^2}{R_b^2}} \sqrt{R_b^2 - \frac{C_i^2}{R_b^2}},$$

where W_i has the first-coordinate law on the sphere S^{n-2}. Equivalently,

$$\frac{W_i + 1}{2} \sim \mathrm{Beta}\left(\frac{n-2}{2}, \frac{n-2}{2}\right).$$

For $x \in [-1, 1]$, write

$$T_n(x) := \frac{1}{2} I_{1-x^2}\left(\frac{n-2}{2}, \frac{1}{2}\right) = \Pr[W_i > x],$$

and extend T_n by $T_n(x) = 1$ for $x \leq -1$ and $T_n(x) = 0$ for $x \geq 1$. Under Heuristic 2,

$$\Pr\big[|(ab)_{[i]}| > \Delta \,\big|\, b, R_a, S_0, C_i\big] \approx T_n\left(\frac{\Delta - \mu_i}{\sigma_i}\right) + T_n\left(\frac{\Delta + \mu_i}{\sigma_i}\right). \tag{14}$$

This approximation is coefficient-side and conditions on the actual shift-correlation value $C_i = (b\bar{b})_{[i]}$, rather than on the spectral envelope $\Lambda(A)$ used in the certified covariance bound. Let $E_j := \{|(ab)_{[j]}| > \Delta\}$. Under trimming, define

$$p_{\text{trim}} := \Pr(E_0 \mid \|(b\bar{b})_{[1:n/2)}\|_\infty \leq B_{\text{inner}})$$

and, for any fixed $i \neq 0$,

$$q_{\text{trim}} := \Pr(E_0 \wedge E_i \mid \|(b\bar{b})_{[1:n/2)}\|_\infty \leq B_{\text{inner}}).$$

Using (14), we approximate

$$q_{\text{trim}} \approx \mathbb{E}\left[\mathbf{1}_{E_0}\left(T_n\left(\frac{\Delta - \mu_i}{\sigma_i}\right) + T_n\left(\frac{\Delta + \mu_i}{\sigma_i}\right)\right) \,\middle|\, \|(b\bar{b})_{[1:n/2)}\|_\infty \leq B_{\text{inner}}\right]. \tag{15}$$

Here μ_i and σ_i are computed from the conditioned values of b, R_a, $S_0 = (ab)_{[0]}$, and $C_i = (b\bar{b})_{[i]}$. As a second heuristic step, we approximate the exceedance indicators by an exchangeable Bernoulli family with the same first two moments, which yields a beta-binomial model for N_Δ. Define

$$\rho_{\text{trim}} := \frac{q_{\text{trim}} - p_{\text{trim}}^2}{p_{\text{trim}}(1 - p_{\text{trim}})},$$

and, when $0 < \rho_{\text{trim}} < 1$, set

$$\alpha_{\text{trim}} := p_{\text{trim}}(\rho_{\text{trim}}^{-1} - 1), \qquad \beta_{\text{trim}} := (1 - p_{\text{trim}})(\rho_{\text{trim}}^{-1} - 1).$$

This leads to the beta-binomial closure of DFR estimate as

$$\text{DFR}_{\text{dep,heur}} := \sum_{s=t+1}^{n} \binom{n}{s} \frac{\text{Beta}(s + \alpha_{\text{trim}},\, n - s + \beta_{\text{trim}})}{\text{Beta}(\alpha_{\text{trim}},\, \beta_{\text{trim}})}, \tag{16}$$

where $\text{Beta}(\cdot, \cdot)$ denotes the Beta function, not the Beta distribution.

5.1.3 Validation Experiments The roles of the two parts are therefore distinct. Let $\text{DFR}_{\text{dep,upper}}$ be the theorem-backed quantity of the certified worst-case upper bound from Equation (13), and $\text{DFR}_{\text{dep,heur}}$ be the shaper but heuristic upper bound from Equation (16).

In the case-study tables below, we therefore reserve the notation $\mathrm{DFR}_{\mathrm{dep,upper}}$ for rows obtained by plugging measured spectral envelopes into Equation (13). By contrast, the three-vector model is a sharper but heuristic approximation that explains how trimming changes the local quantities μ_i and σ_i and accounts for the observed tail clustering. We then validate the rigorous upper bound and the accuracy of the heuristic estimate.

We validate the revised Section 5.1 in two regimes, each targeting a different logical component rather than concrete cryptographic rare-event magnitudes. First, we use an exact-support toy model in which the left factor a is sampled uniformly from the fixed-weight distribution on $\mathcal{R}_6$ with weight 2, the right factor b has i.i.d. ternary coefficients with $\Pr[b_i = \pm 1] = 0.05$, and the DFR event is $N_1 > 1$ for $z = ab$. In this setting the accepted set can be enumerated exactly, so Λ_{trim} is computed exactly rather than estimated. Table 1 reports the exact DFR and the certified bound for two trim levels. As expected, the theorem-backed bound is conservative, but it moves in the correct direction when the trim rule is strengthened.

Table 1: Exact-support validation of the certified trimmed DFR bound for the toy model $(n, w, p, \Delta, t) = (6, 2, 0.1, 1, 1)$

B_{inner}	pass rate	Λ_{trim}	exact DFR	certified bound
0	0.20	0.20	2.98×10^{-4}	4.04×10^{-1}
1	1.00	0.40	1.09×10^{-3}	1

Second, we use a moderate-dimension fixed-weight model to validate the heuristic layer. Here a is sampled from the fixed-weight distribution on $\mathcal{R}_{32}$ with weight 8, b again has i.i.d. ternary coefficients with $\Pr[b_i = \pm 1] = 0.1$, and the evaluation event is $N_1 > 3$ for $z = ab$. This is not a concrete cryptographic parameter regime; it is a held-out stress test chosen so that the heuristic can be compared against observable frequencies on disjoint calibration and evaluation data. A calibration split of 5000 samples sets the trim cutoffs at the empirical 33rd and 66th percentiles of the inner-distortion score, namely $B_{\mathrm{inner}} = 2$ and $B_{\mathrm{inner}} = 3$, and a disjoint evaluation split of 20000 samples is used for the reported numbers. Table 2 compares the empirical DFR and empirical one-coordinate tail against the three-vector heuristic and its optional Poisson-binomial closure. The heuristic remains conservative, but it preserves the correct ordering between trim levels and is substantially sharper than the certified theorem bound in this regime.

The same evaluation split also shows that the trimming statistic tracks tail clustering. Using the proxy cluster event $C_{3,2}(ab) = \mathbf{1}[T_3(ab) \geq 2]$, where $T_3(ab)$ counts coefficients above the intermediate cutoff 3, the observed cluster rate increases monotonically across the low/middle/high bins of the inner-distortion score; see Table 3. The mean number of exceedances stays nearly constant, so the

Table 2: Moderate-dimension validation of the three-vector heuristic on the calibrated trim levels

trim level	pass rate	empirical DFR	heuristic DFR	empirical single tail	heuristic single tail
$q_{0.33}$ ($B_{\text{inner}} = 2$)	0.4296	0.8374	0.9860	0.2160	0.4179
$q_{0.66}$ ($B_{\text{inner}} = 3$)	0.9046	0.8296	0.9859	0.2167	0.4189

statistic is capturing clustering rather than merely a shift in the one-coordinate marginal.

Table 3: Observed tail clustering versus bins of the controlling trimming statistic

score bin	samples	mean $T_3(ab)$	cluster rate $\Pr[C_{3,2}(ab) = 1]$
low ($\|(a\bar{a})_{[1:n/2)}\|_\infty \leq 2$)	8680	0.2059	0.0300
middle ($= 3$)	9496	0.2060	0.0312
high (≥ 4)	1824	0.2050	0.0329

Finally, because the generic predicate IsDistorted can be calibrated by a proxy sampler, we compare the exact fixed-weight sampler against the Gaussian proxy with matched per-coordinate variance $w/n = 0.25$. Table 4 shows that the proxy has a visibly heavier inner-distortion tail: its 90th percentile is 3.98 whereas the exact sampler's 90th percentile is 3.0. When the proxy's 90th-percentile threshold is applied to a fresh evaluation split, it accepts 91.34% of exact samples versus 89.95% of proxy samples. This gap is small enough for coarse threshold selection, but large enough that the proxy should not be mistaken for an exact replacement of the fixed-weight law.

Table 4: Exact-sampler versus Gaussian-proxy calibration for the inner-distortion statistic

sampler	mean	$q_{0.90}$	$q_{0.95}$	$q_{0.99}$	accept at proxy $q_{0.90}$
exact fixed weight	2.6524	3.0	4.0	4.0	0.9134
Gaussian proxy	2.7017	3.9776	4.4581	5.3944	0.8995

5.2 TrimPKE

5.2.1 Framework Description We begin by introducing an auxiliary algorithm IsDistorted, which arises from Equation (9) and evaluates both the *inner*

distortion and *outer distortion* of a polynomial. Based on this evaluation, we define two rejection-sampling procedures: Reject_{key} and Reject_{enc}. Figure 8 shows the variant tailored for Module-LWE-based encryptions, and Figure 9 shows the variant designed for NTRU-based encryptions. The notation is consistent with the framework in Figures 1 and 2. Within this framework, let ρ_0 denote the initial seed used to generate randomness during encryption. Let E be a variable-length input hash function. While it may seem more natural to resample a fresh seed ρ_0, in the construction like KEMs, KEs, or AKEs based on PKEs, the ciphertext must remain deterministic with respect to the same seed ρ_0. To achieve this, we repeatedly derive new seeds through the random oracle, thereby ensuring that the same input ρ_0 always yields the same ciphertext.

Fig. 7: **Generic TrimPKE Framework**

$\mathsf{TrimGen}$	$\mathsf{TrimEnc}(pk, m, \rho_0)$
1 : $(pk, sk) \leftarrow \mathsf{KeyGen}$	1 : $N \leftarrow 0$
2 : **if** $\mathsf{Reject}_{key}(pk, sk) = 0$	2 : $\rho \leftarrow \rho_0$
3 : go back to step 1	3 : $\rho \leftarrow \mathsf{E}(N, \rho \| \rho_0)$
4 : **return** (pk, sk)	4 : $c \leftarrow \mathsf{Encrypt}(pk, m, \rho)$
	5 : **if** $\mathsf{Reject}_{enc}(pk, m, \rho, c) = 0$
$\mathsf{TrimDec}(c, sk)$	6 : $N \leftarrow N + 1$
1 : $m \leftarrow \mathsf{Decrypt}(c, sk)$	7 : go back to step 3
2 : **return** m	8 : **return** c

We denote by $\sigma_s, \sigma_e, \sigma_r, \sigma_{e_1}, \sigma_g, \sigma_f, \sigma_t$ the standard deviations of the coefficient distributions $\mathcal{X}_s, \mathcal{X}_e, \mathcal{X}_r, \mathcal{X}_{e_1}, \mathcal{X}_g, \mathcal{X}_f, \mathcal{X}_t$ of the corresponding polynomials. We align $\omega_{\text{outer}}, \omega_{\text{inner}}$ into one single value, and denote by $\omega_s, \omega_e, \omega_r, \omega_{e_1}, \omega_g, \omega_f, \omega_t$ the trim parameters of polynomials. For each sampler, let $\theta_a = (\mu^{(a)}_{\text{outer}}, \sigma^{(a)}_{\text{outer}}, \mu^{(a)}_{\text{inner}}, \sigma^{(a)}_{\text{inner}})$ denote the calibration tuple used by the predicate. The Gaussian-surrogate formulas of Section 3.2 provide one possible choice for θ_a, but concrete instantiations may instead use exact or empirical sampler moments. Let $\delta_s, \delta_e, \delta_r, \delta_{e_1}, \delta_g, \delta_f, \delta_t$ denote the pass probabilities on these calibrated predicates, where $\delta_s := \Pr_{a \leftarrow \mathcal{X}_s^n}[\mathsf{IsDistorted}((a), \theta_s, \omega_s) = 0]$ denotes the single-polynomial pass probability; for k-dimensional vectors with independent components, the pass probability is δ_s^k.

We define the probabilities that Reject_{key} and Reject_{enc} output 0 as r_{key} and r_{enc}, respectively. For Module-LWE-based rejection, we obtain

$$\begin{aligned} r_{\text{key}} &= 1 - \delta_s^k \cdot \delta_e^l, \\ r_{\text{enc}} &= 1 - \delta_r^l \cdot \delta_{e_1}^k. \end{aligned} \tag{17}$$

Algorithm 1 IsDistorted

Input: vector $\boldsymbol{a} \in \mathcal{R}_n^k$, calibration tuple $\theta_a = (\mu_{\text{outer}}, \sigma_{\text{outer}}, \mu_{\text{inner}}, \sigma_{\text{inner}})$, trim parameter ω

Output: distorted or not $b \in \{0, 1\}$

```
b ← 0
for i from 0 to k − 1
    if |‖a_[i]‖_2^2 − μ_outer| > ω · σ_outer or |‖(a_[i] ā_[i])_[1:n/2)‖_∞ − μ_inner| > ω · σ_inner
        b ← 1
return b
```

For NTRU-based rejection, we have

$$r_{\text{key}} = 1 - \delta_g \cdot \delta_f, \quad r_{\text{enc}} = 1 - \delta_t. \tag{18}$$

We include (pk, m, c) in the formal interface of Reject_{enc} to accommodate compression-aware variants, such as Kyber [6] and DAWN [42], where the public ciphertext and public inputs may be needed to inspect a polynomial created by compression. The pass-rate formulas in Equations (17) and (18) refer to the basic setting discussed above, where the accept/reject decision depends only on the sampled polynomials. Concrete compressed instantiations may add extra public-input checks without changing the surrounding reduction interface.

5.2.2 Impact on Concrete Security We now isolate the concrete-security effect that is actually analyzed in this paper. Our predicates are public functions with public parameters, but in key generation, they are evaluated on secret samples. We therefore do *not* claim a general timing-side-channel theorem here. Instead, the point relevant to the reduction is narrower: trimming changes the output distribution by rejecting some samples, and that support loss must be charged explicitly in the concrete security bounds below. This is analogous to other public-predicate rejection procedures used in lattice cryptography, such as Falcon key generation [23], but any implementation-level leakage analysis is outside the scope of the present work.

The concrete cryptographic effect of trimming is therefore not leakage, but a reduction of the effective sample space, which must be reflected in the hardness assumptions and concrete-security bounds. We summarize the main assumptions underlying current module-lattice-based encryption schemes as follows: Ring-LWE [45], Module-LWE [6], Ring-LWR [24], Module-LWR [16], and NTRU [30]. While these problems differ in algebraic structure, their state-of-the-art concrete-security estimates are broadly comparable, and our security analysis accounts explicitly for the loss induced by rejection.

For the power-of-two cyclotomic settings and parameter ranges used in practice, the best known attacks against Ring/Module-LWE and Ring/Module-LWR are still lattice-reduction based and are typically estimated via the same BKZ-style cost models as for LWE. Likewise, no attack is known that gains an advantage from the rounding operation in Ring-LWR or Module-LWR. NTRU

Fig. 8: **Module-LWE-based Rejection**

$\mathsf{Reject}^{\mathsf{MLWE}}_{key}(pk := (\mathbf{A}, \boldsymbol{b}), sk := \boldsymbol{s})$

$rej \leftarrow 1$
if $\mathsf{IsDistorted}(\boldsymbol{s}, \theta_s, \omega_s) = 1$
  $rej \leftarrow 0$
$\boldsymbol{e} \leftarrow \boldsymbol{b} - \boldsymbol{s}\mathbf{A}^{\mathsf{T}} \pmod q$
if $\mathsf{IsDistorted}(\boldsymbol{e}, \theta_e, \omega_e) = 1$
  $rej \leftarrow 0$
return rej

$\mathsf{Reject}^{\mathsf{MLWE}}_{enc}(pk, m, \rho := (\rho_r, \rho_{e_1}), c)$

$\boldsymbol{r} \leftarrow \mathsf{Sample}((\mathcal{X}_r^n)^l, \rho_r)$
$\boldsymbol{e}_1 \leftarrow \mathsf{Sample}((\mathcal{X}_{e_1}^n)^k, \rho_{e_1})$
$rej \leftarrow 1$
if $\mathsf{IsDistorted}(\boldsymbol{r}, \theta_r, \omega_r) = 1$
  $rej \leftarrow 0$
if $\mathsf{IsDistorted}(\boldsymbol{e}_1, \theta_{e_1}, \omega_{e_1}) = 1$
  $rej \leftarrow 0$
return rej

Fig. 9: **NTRU-based Rejection**

$\mathsf{Reject}^{\mathsf{NTRU}}_{key}(pk := h, sk := f)$

$g \leftarrow hf \pmod q$
$rej \leftarrow 1$
if $\mathsf{IsDistorted}(g, \theta_g, \omega_g) = 1$
  $rej \leftarrow 0$
$f' \leftarrow (f - 1)/p$
if $\mathsf{IsDistorted}(f', \theta_f, \omega_f) = 1$
  $rej \leftarrow 0$
return rej

$\mathsf{Reject}^{\mathsf{NTRU}}_{enc}(pk, m, \rho, c)$

$t \leftarrow \mathsf{Sample}(\mathcal{X}_t^n, \rho)$
$rej \leftarrow 1$
if $\mathsf{IsDistorted}(t, \theta_t, \omega_t) = 1$
  $rej \leftarrow 0$
return rej

instances are closely related structured lattice problems except for the dense-sublattice attack [17]; modern parameter sets account for this in their security estimates. However, this attack primarily affects parameter settings with large moduli; modern NTRU-based schemes account for this in their parameter choices and therefore effectively mitigate the threat. Consequently, most schemes treat Ring-LWE, Module-LWE, Ring-LWR, Module-LWR, and NTRU as instances of LWE and rely on the lattice estimator[2] from [1] to evaluate the security of the corresponding LWE or NTRU instance, which is then taken as the concrete security level of the scheme.

In what follows, we review the mainstream methods for solving LWE or NTRU and explain how our rejection sampling technique affects concrete security through explicit support-loss terms and, empirically, much smaller estimator

[2] https://github.com/malb/lattice-estimator

shifts. For a LWE instance $\mathsf{LWE}_{m,n,q,\mathcal{X}_s,\mathcal{X}_e}$, we have $\boldsymbol{s}\mathbf{A}^\intercal + \boldsymbol{e} \equiv \boldsymbol{b} \pmod q$, where $\mathbf{A} \in \mathbb{Z}^{m\times n}$, and the entries of $\mathbf{A}$ are drawn from the uniform distribution $\mathcal{U}(\mathbb{Z}_q)$, and the entries of $\boldsymbol{s}, \boldsymbol{e}$ are drawn from the distributions $\mathcal{X}_s$ and $\mathcal{X}_e$ whose the standard deviations are σ_s, σ_e, respectively. The mainstream methods for solving LWE are lattice-based attacks, which can be broadly categorized into two types: primal and dual attacks.

Primal Attack. To solve LWE, we can construct the lattice

$$\mathcal{L}_{\mathbf{A},q} = \mathbb{Z}^{(m+n)\times(m+n)} \cdot \begin{pmatrix} \mathbf{A}^\intercal & \mathbf{I}_n \\ q\cdot\mathbf{I}_m & \mathbf{0} \end{pmatrix}.$$

For a fixed target vector $\boldsymbol{v} = (\boldsymbol{b}, \mathbf{0})$, there exists a nearby lattice vector $\boldsymbol{v} - (\boldsymbol{e}, -\boldsymbol{s})$, with gap vector $(\boldsymbol{e}, -\boldsymbol{s})$, which is extremely short. The distribution of $\mathcal{X}_s, \mathcal{X}_e$ guarantees that $\boldsymbol{v}$ is the unique closest vector with overwhelming probability. Thus, the problem reduces to solving a Bounded Distance Decoding (BDD) instance over $\mathcal{L}_{\mathbf{A},q}$. Depending on the method, the primal attack has two main branches: primal BDD and primal unique SVP (uSVP).

- For primal BDD, we apply BKZ reduction [48,49] to obtain a reduced basis of $\mathcal{L}_{\mathbf{A},q}$, and then solve the BDD instance using Babai's rounding or nearest-plane algorithms.
- For primal uSVP, we use the embedding technique [36,3] and build the extended lattice:

$$\mathcal{L}_{\mathbf{A},\boldsymbol{b},q} = \mathbb{Z}^{(m+n+1)\times(m+n+1)} \cdot \begin{pmatrix} \mathbf{A}^\intercal & \mathbf{I}_n & 0 \\ q\cdot\mathbf{I}_m & \mathbf{0} & 0 \\ \boldsymbol{b} & \mathbf{0} & 1 \end{pmatrix},$$

  and then apply lattice reduction directly to solve the unique SVP instance, thereby recovering $\boldsymbol{s}$ and $\boldsymbol{e}$.

Dual Attack. Alternatively, we partition $\mathbf{A}$ into two submatrices $\mathbf{A_0} \in \mathbb{Z}^{m\times n_0}$ and $\mathbf{A_1} \in \mathbb{Z}^{m\times n_1}$, and split $\boldsymbol{s}$ into $\boldsymbol{s_0} \in \mathbb{Z}^{n_0}$ and $\boldsymbol{s_1} \in \mathbb{Z}^{n_1}$ with $n_0 + n_1 = n$. Thus,

$$\boldsymbol{s_0}\mathbf{A_0}^\intercal + \boldsymbol{s_1}\mathbf{A_1}^\intercal + \boldsymbol{e} \equiv \boldsymbol{b} \pmod q.$$

We construct the lattice as:

$$\mathcal{L}_{\mathbf{A},q,n_0} = \mathbb{Z}^{(m+n_0)\times(m+n_0)} \cdot \begin{pmatrix} \mathbf{A_0} & \mathbf{I}_m \\ q\cdot\mathbf{I}_{n_0} & \mathbf{0} \end{pmatrix}.$$

By applying lattice reduction to $\mathcal{L}_{\mathbf{A},q,n_0}$, we obtain multiple short vectors $(\boldsymbol{w}, \boldsymbol{u})$ satisfying $\boldsymbol{u}\mathbf{A_0} \equiv \boldsymbol{w} \pmod q$. This leads to

$$\boldsymbol{b}\boldsymbol{u}^\intercal - \boldsymbol{s_1}\mathbf{A_1}\boldsymbol{u}^\intercal \equiv \boldsymbol{s_0}\boldsymbol{w}^\intercal + \boldsymbol{e}\boldsymbol{u}^\intercal \pmod q.$$

Since both $\boldsymbol{s_0}\boldsymbol{w}^\intercal$ and $\boldsymbol{e}\boldsymbol{u}^\intercal$ are small, we can exhaust over possible values of $\boldsymbol{s_1}$, and then iteratively recover the full secret $\boldsymbol{s}$.

From the above, the cost of attacks depends mainly on n, q, and the standard deviations σ_s, σ_e. Trimming changes this picture in two conceptually different ways. First, it changes the output distribution by shrinking the support, which is captured information-theoretically by the pass probabilities $\delta_s, \delta_e, \delta_r, \delta_{e_1}, \delta_g, \delta_f, \delta_t$ of $\mathsf{IsDistorted}$. Second, it may shift the effective standard deviations seen by current lattice estimators. We observed only small estimator shifts in exploratory runs, but we do not rely on a particular percentage here and keep the formal discussion at the support-loss level. Beyond pure lattice algorithms, state-of-the-art attacks also combine lattice reduction with combinatorial techniques, such as Meet-in-the-Middle (MITM). The complexity of such hybrid attacks depends on the effective search space for $\boldsymbol{s}$ and $\boldsymbol{e}$. Accordingly, for a Module-LWE-based encryption scheme, the support reduction alone can weaken the concrete security of the private key by at most $l \cdot \log_2(\delta_e^{-1}) + k \cdot \log_2(\delta_s^{-1})$ bits, while the security of the message can be weakened by at most $l \cdot \log_2(\delta_r^{-1}) + k \cdot \log_2(\delta_{e_1}^{-1})$ bits. For an NTRU-based encryption scheme, the corresponding support-loss terms are at most $\log_2(\delta_g^{-1}) + \log_2(\delta_f^{-1})$ bits for the private key and $\log_2(\delta_t^{-1})$ bits for the message. Concretely, for the instantiations used in later $\mathsf{TrimLAC}$ and $\mathsf{TrimDAWN}$, the pass probabilities are at least $\frac{1}{2}$, therefore leading to at most 2 bits loss in concrete security.

It is worth noting that, for many parameter sets, hybrid attacks with MITM are not optimal, so the above constitutes only a worst-case analysis. Moreover, our rejection sampling technique filters out polynomials with abnormally small norms, thereby strengthening concrete security in specific scenarios.

5.2.3 Implementation and Acceleration The core component of our framework is the algorithm $\mathsf{IsDistorted}$, which requires computing the product of a polynomial a with its conjugate $\bar{a}$. To achieve efficiency, we employ the Number Theoretic Transform (NTT), reducing polynomial multiplication in $\mathcal{R}_n/Q\mathcal{R}_n$ from $\mathcal{O}(n^2)$ to $\mathcal{O}(n \log n)$.

Let n be a power of two and ζ a primitive $2n$-th root of unity. Then

$$\mathbb{Z}_Q[x]/(x^n+1) \;\cong\; \mathbb{Z}_Q[x]/\big(x^{\frac{n}{2}} - \zeta^{\frac{n}{2}}\big) \times \mathbb{Z}_Q[x]/\big(x^{\frac{n}{2}} + \zeta^{\frac{n}{2}}\big).$$

If $\mathbb{Z}_Q$ lacks such a primitive root, we instead use a $\frac{2n}{d}$-th root, which yields $\frac{n}{d}$ irreducible factors of degree d. The transform is *complete* when $d = 1$ and *incomplete* when $d > 1$. Moreover, the NTT of $\bar{a}$ can be derived directly from a, even in the incomplete case. Specifically, we have

$$a \equiv \sum_{i=0}^{d-1} a_{[i]} x^i \pmod{x^d - \zeta}, \quad \bar{a} \equiv \sum_{i=0}^{d-1} \zeta\, a_{[i]}\, x^{d-1-i} \pmod{x^d - \zeta^{-1}}.$$

This property eliminates one full NTT computation, further improving efficiency. We note that the standard NTT only calculates the result in $\mathbb{Z}_Q$, and we need the result of $a\bar{a}$ in $\mathbb{Z}$. However, we can use the NTT lifting techniques in [13,43], which use a large enough modulus Q to ensure that all the coefficients in the polynomial product would not exceed $\lfloor \frac{Q}{2} \rfloor$ or with a negligible probability.

5.3 TrimKEM

By applying the FO transformation to TrimPKE, we obtain a generic TrimKEM framework as shown in Figure 10, which can be used to construct Trim-protected KEM schemes.

Fig. 10: **Generic TrimKEM Framework**

$\mathsf{TrimGen}^{\not\perp}$

1 : $(pk, sk') \leftarrow \mathsf{TrimGen}$
2 : $\bar{K}' \leftarrow \mathsf{Sample}(\mathcal{M})$
3 : $sk \leftarrow \big(sk', pk, \mathsf{H}(pk), \bar{K}'\big)$
4 : **return** (pk, sk)

$\mathsf{TrimEnca}\,(pk)$

1 : $m \leftarrow \mathsf{Sample}(\mathcal{M})$
2 : $(\bar{K}, \rho_0) \leftarrow \mathsf{G}(m, \mathsf{H}(pk))$
3 : $c \leftarrow \mathsf{TrimEnc}(pk, m, \rho_0)$
4 : $K \leftarrow \mathsf{F}(\bar{K}, c)$
5 : **return** (c, K)

$\mathsf{TrimDeca}\,\big(c, sk = \big(sk', pk, \mathsf{H}(pk), \bar{K}'\big)\big)$

1 : $m' \leftarrow \mathsf{TrimDec}(c, sk')$
2 : $(\bar{K}, \rho) \leftarrow \mathsf{G}(m', \mathsf{H}(pk))$
3 : **if** $c \neq \mathsf{TrimEnc}(pk, m', \rho)$
4 : $\quad K \leftarrow \mathsf{F}(\bar{K}', c)$
5 : **else**
6 : $\quad K \leftarrow \mathsf{F}(\bar{K}, c)$
7 : **return** K

5.4 TrimLAC and TrimDAWN

Because Module-LWE-based and NTRU-based schemes differ in how trimming is most naturally integrated, we present two concrete instantiations of the TrimPKE and TrimKEM frameworks. For the Module-/Ring-LWE side, we construct a trimmed variant of LAC-v3a in [45], denoted TrimLAC. For the NTRU side, we construct a trimmed variant of DAWN-α in [42], denoted TrimDAWN. (Strictly speaking, LAC is Ring-LWE-based; we treat it as a special case of Module-LWE, and its use of ECC makes it a representative testbed for our framework.)

For TrimLAC, we follow the latest reference implementation in the repository[3]. LAC samples from fixed-Hamming-weight distributions, which already impose an implicit rejection on *outer* distortion. Accordingly, in TrimLAC we focus trimming on *inner* distortion only, by enforcing an explicit upper bound B_{inner} on $\|(a\bar{a})_{[1:n/2)}\|_\infty$ (and analogously for b). Since the outer behavior is effectively fixed by the sampling rule, this avoids the need to additionally control skewness in the squared ℓ_2-norm. For this reason, in Algorithms 2-3 we do not invoke the generic predicate IsDistorted; instead, we directly compute and check

[3] https://github.com/pqc-lac/lac-intel64

$\|(a\bar{a})_{[1:n/2)}\|_\infty$. Complete algorithms for both TrimLAC and TrimDAWN are provided in Appendix A. For the DFR discussion, the only point we use is that the reseeding procedure is a public deterministic transform of the seed and public inputs. Accordingly, the case-study analysis below reports the measured rejection rates and the resulting tail metrics under the implemented resampling rule, rather than relying on a separate hash-uniformity theorem.

5.4.1 Performance Comparison We report performance and dependency evaluations of TrimLAC and TrimDAWN under varying trim parameters in Table 5. For TrimLAC, trimming is parameterized by an upper bound B_{inner} on the inner-distortion infinity norm; for TrimDAWN, the reported entries use the instantiated predicate $\mathsf{IsDistorted}(\cdot, \theta_\bullet, \omega)$ with scheme-specific calibration tuples $\theta_\bullet$ and a scalar parameter ω. Thus every pass rate, runtime overhead, and tail metric below is attached to the actual predicate used by the named scheme, not to an abstract cross-scheme proxy. The choice $\omega = \infty$ (or $B_{\text{inner}} = \infty$) recovers the original scheme without rejection sampling and serves as the baseline. Importantly, every trimmed row in this table should be read as a case-study model value associated with the named predicate, not as a theorem-backed sampler-level guarantee. Table 5 is an implementation snapshot rather than a formal benchmark artifact; none of the theorem-backed claims of Sections 3–5.1 depend on these concrete rows.

We report runtime costs as tuples in kcycles. For example, $(27, 46, 62)$ denotes the kcycle costs of key generation, encapsulation, and decapsulation for TrimLAC at NIST level I with no trimming. We also report the trim bounds B_{inner} as tuples across security levels. For example, $(36, 36, 40)$ denotes the values of B_{inner} used at NIST-I, III, and V, respectively.

To quantify the residual tail risk after trimming dependency outliers, we separate two quantities. For trimmed settings, we report the instantiated inner-distortion-control metric $\mathrm{DFR}_{\mathrm{dep,heur}}$ from Equation (16). Operationally, each trimmed row is produced in two stages: we first obtain a numerical estimate of the relevant inner-distortion-control input from accepted-sample statistics under the named scheme and predicate, and we then plug that input into Equation (13). In Table 5, these are model values attached to the chosen predicates rather than certified sampler-level guarantees. In the baseline setting $\omega = \infty$ (equivalently, $B_{\text{inner}} = \infty$), finite data do not certify the full worst-case quantity, so we instead report a truncated empirical proxy $\widetilde{\mathrm{Tail}}_{\mathrm{dep}}$ obtained from dependency events whose empirical probability is at least 2^{-16}. This restriction is conservative in the following sense: an adversary may invest additional offline effort to search for rarer ciphertexts with stronger dependencies, in which case trimming can provide an even larger improvement against worst-case ciphertext selection than what this baseline proxy captures. The baseline rows and the trimmed rows therefore answer different questions and should not be read as one numerically comparable estimator family.

In TrimDAWN, the instantiated inner-distortion-control quantity drops from the high-2^{-100} range to below 2^{-133} (resp. 2^{-155}), with an overhead of approxi-

mately 31%–37%. In TrimLAC, the corresponding metric drops below 2^{-109} at an overhead of about 85%, whereas pushing it below 2^{-121} incurs prohibitive cost (up to 1982% in our evaluation). The untrimmed LAC baseline remains only a low-2^{-30} truncated proxy, not a worst-case statement; the NIST-V baseline entry reaches the truncation floor and therefore appears as 0 in Table 5. This disparity reflects the underlying decoding regimes: LAC is designed to correct many errors (roughly 8–20), while DAWN corrects only a single error. Consequently, the probability of observing a single above-threshold coefficient is much larger in LAC (above 2^{-20}) than in DAWN (below 2^{-70}), giving dependency amplification substantially more opportunities to affect the decoding outcome. Finally, in the Module-LWE setting, the decryption noise often aggregates multiple (approximately) independent polynomial products, which further attenuates convolution-induced correlations and can make trimming more effective in practice.

Table 5: Performance, baseline dependency-tail proxies, and instantiated inner-distortion-control metrics for TrimLAC and TrimDAWN. Baseline rows report truncated empirical proxies. All trimmed rows report case-study model values obtained from Equation (13); none should be read as a theorem-backed sampler-level guarantee.

Trim B_{inner}		TrimLAC			Trim ω	TrimDAWN	
		NIST-I	NIST-III	NIST-V		NIST-I	NIST-V
$+\infty$	Cost	(27, 46, 61)	(41, 69, 99)	(58, 105, 186)	$+\infty$	(23, 20, 20)	(46, 36, 38)
	$\log_2\left(\widetilde{\text{Tail}}_{\text{dep}}\right)$	-32	-169	0		-98	-145
(36, 32, 40)	Cost	(34, 57, 75)	(54, 78, 115)	(79, 136, 225)	5	(25, 26, 27)	(54, 50, 61)
	$\log_2(\text{DFR}_{\text{dep,heur}})$	-97	-235	-149		-126	-155
(32, 28, 36)	Cost	(48, 91, 109)	(58, 83, 121)	(116, 229, 325)	2	(26, 28, 29)	(59, 55, 64)
	$\log_2(\text{DFR}_{\text{dep,heur}})$	-109	-258	-191		-133	-160
(28, 24, 32)	Cost	(185, 1295, 1311)	(85, 160, 192)	(547, 6121, 6131)	1	(30, 52, 45)	(79, 111, 104)
	$\log_2(\text{DFR}_{\text{dep,heur}})$	-121	-282	-235		-133	-162

6 Security Proofs of TrimPKE and TrimKEM

In this section, based on the security of the public-key encryption scheme PKE, we prove the QROM security of the previously defined TrimPKE and TrimKEM. We use $\mathcal{T}_{\mathcal{A}}$ to denote the time cost of the algorithm $\mathcal{A}$.

6.1 IND-CPA Security Proof of TrimPKE in the QROM

Theorem 2. *Let $\mathcal{A}$ be an* IND-CPA *adversary against* TrimPKE*, making q_{E} quantum queries to the random oracle* E*. Then we can construct an* IND-CPA *adversary $\mathcal{B}$ against* PKE *such that $\mathcal{T}_{\mathcal{B}} \approx \mathcal{T}_{\mathcal{A}}$ and*

$$\text{Adv}^{\text{IND-CPA}}_{\text{TrimPKE},\mathcal{A}} \leq 4st \cdot \text{Adv}^{\text{IND-CPA}}_{\text{PKE},\mathcal{B}} + 2s \cdot (r_{enc})^t + (r_{key})^s + \frac{2q_{\mathsf{E}}}{\sqrt{|\mathcal{R}|}}$$

holds for any positive integers s and t. Here r_{key} and r_{enc} are defined in Eq. (17), and $\mathcal{R}$ is the randomness space of PKE.

Before giving the proof, we first point out that the upper bound given by this theorem holds for all positive integers s and t; thus, our bound remains reasonable as long as both r_{enc} and r_{key} are relatively small. One may wonder why the bound here holds for all positive integers s and t. Taking s as an example, intuitively, our proof idea is as follows: first, we set the upper bound on the number of rejection sampling of the key generation algorithm to s; then, we uniformly select a (pk, sk) pair from at most s such pairs, and guess that this pair is the one used for the challenge ciphertext. It can be observed that this process is independent of the specific value of s, and it only needs s to be a positive integer. In fact, the larger s is, the lower $(r_{key})^s$ becomes; however, the success probability of the random guess we make also decreases, i.e., the value of "$4st \cdot \mathrm{Adv}^{\mathsf{IND\text{-}CPA}}_{\mathsf{PKE},\mathcal{B}}$" increases. Thus, the values of s and t can be appropriately chosen based on the magnitudes of r_{key} and r_{enc}.

Proof. We introduce a sequence of games $\mathbf{G_0}$ to $\mathbf{G_{11}}$ as shown in Figure 11 and Figure 12 to prove this theorem, and the game $\mathbf{G_0}$ is the $\mathsf{IND\text{-}CPA}$ game of $\mathsf{TrimPKE}$. Hence we have

$$\mathrm{Adv}^{\mathsf{IND\text{-}CPA}}_{\mathsf{TrimPKE},\mathcal{A}} = \Pr\left[1 \leftarrow \mathbf{G_0}\right]. \tag{19}$$

Game $\mathbf{G_1}$: This game is the same as game $\mathbf{G_0}$, except that the encryption algorithm $\mathsf{TrimEnc}$ is replaced by $\mathsf{TrimEnc_1}$. Compared with the $\mathsf{TrimEnc}$ defined in Figure 7, $\mathsf{TrimEnc_1}$ uniformly samples r to encrypt m and no longer derive it by querying E.

As shown in Figure 7, when deriving the randomness r, the $\mathsf{TrimEnc}$ introduces a counter k and imposes that all the input to E end with r_0. Therefore, if $\mathcal{A}$ has never queried E with inputs of the form $*||r_0$, the randomness derived by E in $\mathsf{TrimEnc}$ is uniformly random in $\mathcal{A}$'s view, and thus $\mathsf{TrimEnc}$ is equivalent to $\mathsf{TrimEnc_1}$. At this point, since $\mathcal{A}$ queries E at most q_{E} times and r_0 is uniformly sampled from $\mathcal{R}$, by the One-Way to Hiding theorem [2] we have

$$\left|\Pr\left[1 \leftarrow \mathbf{G_0}\right] - \Pr\left[1 \leftarrow \mathbf{G_1}\right]\right| \leq \frac{2q_{\mathsf{E}}}{\sqrt{|\mathcal{R}|}}. \tag{20}$$

Game $\mathbf{G_2}$: This game is the same as game $\mathbf{G_1}$, except that the key generation algorithm is replaced by $\mathsf{TrimGen_1}$. Compared with the $\mathsf{TrimGen}$ defined in Figure 7, $\mathsf{TrimGen_1}$ sets the repetition upper bound of the while loop as a positive integer s, i.e., if the (pk, sk) generated by the s-th repetition of KeyGen still results in $\mathsf{Reject}_{key}(pk, sk) = 0$, game $\mathbf{G_2}$ aborts and outputs 0.

Obviously, as long as the event that all the instances of (pk, sk) generated by repeating KeyGen s times result in $\mathsf{Reject}_{key}(pk, sk) = 0$ does not occur, game $\mathbf{G_1}$ and $\mathbf{G_2}$ are identical. Hence, by using the fundamental lemma [50] and the definition of r_{key} given in Eq. (17), we have

$$\left|\Pr\left[1 \leftarrow \mathbf{G_1}\right] - \Pr\left[1 \leftarrow \mathbf{G_2}\right]\right| \leq (r_{\text{key}})^s. \tag{21}$$

GAMES $\mathbf{G_0}$-$\mathbf{G_6}$

$1: (pk, sk) \leftarrow \mathsf{KeyGen}$ $\quad //\mathbf{G_0}\text{-}\mathbf{G_1}$
$\quad (pk, sk) \leftarrow \mathsf{TrimGen}_{i-1}$ $\quad //\mathbf{G_{2 \le i \le 5}}$
$\quad (pk, sk) \leftarrow \mathsf{KeyGen}$ $\quad //\mathbf{G_6}$
$2: (m_0^*, m_1^*) \leftarrow \mathcal{A}^{\mathsf{E}}(pk)$ $\quad //\mathbf{G_0}\text{-}\mathbf{G_5}$
$\quad (m_0^*, m_1^*) \leftarrow \mathcal{A}_1(pk)$ $\quad //\mathbf{G_6}$
$3: b \leftarrow_{\$} \{0,1\}$ $\quad //\mathbf{G_0}\text{-}\mathbf{G_6}$
$\quad c^* \leftarrow \mathsf{TrimEnc}(pk, m_b^*)$ $\quad //\mathbf{G_0}$
$\quad c^* \leftarrow \mathsf{TrimEnc}_1(pk, m_b^*)$ $\quad //\mathbf{G_1}\text{-}\mathbf{G_6}$
$4: b' \leftarrow \mathcal{A}^{\mathsf{E}}(pk, c^*)$ $\quad //\mathbf{G_0}\text{-}\mathbf{G_5}$
$\quad b' \leftarrow \mathcal{A}_1(pk, c^*)$ $\quad //\mathbf{G_6}$
$5:$ **return** $[\![b = b']\!]$ $\quad //\mathbf{G_0}\text{-}\mathbf{G_6}$

$\mathsf{TrimEnc}_1(pk, m)$

$1: r \leftarrow_{\$} \mathcal{R}, c \leftarrow \mathsf{Encrypt}(pk, m, r)$
$2:$ **while** $\mathsf{Reject}_{enc}(pk, m, r, c) = 0$
$\quad r \leftarrow_{\$} \mathcal{R}$
$\quad c \leftarrow \mathsf{Encrypt}(pk, m, r)$
end while
$3:$ **return** c

$\mathsf{TrimGen}_1$

$1: (pk, sk) \leftarrow \mathsf{KeyGen}, n = 1$
$2:$ **while** $\mathsf{Reject}_{key}(pk, sk) = 0$
$\quad \wedge\, n < s$
$\quad n = n + 1$
$\quad (pk, sk) \leftarrow \mathsf{KeyGen}$
end while
$3:$ **if** $\mathsf{Reject}_{key}(pk, sk) = 0$, **abort**
game return 0
else return (pk, sk)

$\mathsf{TrimGen}_3$

$1:$ **for** k **from** 1 **to** s
$\quad (pk_k, sk_k) \leftarrow \mathsf{KeyGen}$
end for
$2:$ **if** Bad **occurs, abort**
game return 0
else
$\quad l \leftarrow_{\$} [s]$
$\quad$ **return** (pk_l, sk_l)

$\mathsf{TrimGen}_2$

$1:$ **for** k **from** 1 **to** s
$\quad (pk_k, sk_k) \leftarrow \mathsf{KeyGen}$
end for
$2:$ **if** Bad **occurs, abort**
game return 0
else return (pk_i, sk_i)

$\mathsf{TrimGen}_4$

$1: c \leftarrow_{\$} \{0,1\}$
$2:$ **for** k **from** 1 **to** s
$\quad (pk_k, sk_k) \leftarrow \mathsf{KeyGen}$
end for
$3:$ **if** $c = 0$, **abort**
game return 0
else
$\quad l \leftarrow_{\$} [s]$
$\quad$ **return** (pk_l, sk_l)

Fig. 11: Games $\mathbf{G_0}$ to $\mathbf{G_6}$ in the proof of Theorem 2.

Game $\mathbf{G_3}$: In this game, the $\mathsf{TrimGen}_1$ is replaced by $\mathsf{TrimGen}_2$, which has the following two differences from $\mathsf{TrimGen}_1$:

1. $\mathsf{TrimGen}_2$ no longer executes a while loop, but instead executes a for loop to generate $(pk_1, sk_1), \ldots, (pk_s, sk_s)$.
2. Let Bad be the event that $(pk_1, sk_1), \ldots, (pk_s, sk_s)$ satisfy $\mathsf{Reject}_{key}(pk_1, sk_1) = \ldots = \mathsf{Reject}_{key}(pk_s, sk_s) = 0$. Then, if Bad occurs, game $\mathbf{G_3}$ aborts and outputs 0. Otherwise, there must exist $1 \le i \le s$ s.t. $\mathsf{Reject}_{key}(pk_i, sk_i) = 1$ and $\mathsf{Reject}_{key}(pk_j, sk_j) = 0$ for any $j < i$, and then $\mathsf{TrimGen}_2$ outputs this (pk_i, sk_i).

It is obvious that $\mathsf{TrimGen}_2$ and $\mathsf{TrimGen}_1$ are equivalent, as $\mathsf{TrimGen}_2$ merely generates all the instances of (pk, sk), which are generated sequentially via a while loop in $\mathsf{TrimGen}_1$, in one go. Hence, we have

$$\Pr[1 \leftarrow \mathbf{G_2}] = \Pr[1 \leftarrow \mathbf{G_3}]. \tag{22}$$

Game $\mathbf{G_4}$: In this game, the $\mathsf{TrimGen}_2$ is replaced by $\mathsf{TrimGen}_3$, which is identical to $\mathsf{TrimGen}_2$ except that if Bad does not occur, it takes (pk_l, sk_l) where $l \leftarrow_{\$} [s]$ as the final return.

By additionally writing out the $l \leftarrow_{\$} [s]$ used by game $\mathbf{G_4}$, we can compute

$$\begin{aligned}
&\Pr[b = 1 : b \leftarrow \mathbf{G_4}, l \leftarrow_{\$} [s]] \\
\overset{(a)}{=} &\Pr[b = 1 \wedge \neg\mathsf{Bad} : b \leftarrow \mathbf{G_4}, l \leftarrow_{\$} [s]] \\
= &\Pr[\neg\mathsf{Bad}] \cdot \Pr[b = 1 : b \leftarrow \mathbf{G_4}, l \leftarrow_{\$} [s] | \neg\mathsf{Bad}] \\
\overset{(b)}{=} &\Pr[\neg\mathsf{Bad}] \cdot \frac{1}{s} \cdot \Pr[b = 1 : b \leftarrow \mathbf{G_4}, l = i | \neg\mathsf{Bad}] \\
&+ \Pr[\neg\mathsf{Bad}] \cdot \frac{s-1}{s} \cdot \Pr[b = 1 : b \leftarrow \mathbf{G_4}, l \neq i | \neg\mathsf{Bad}] \\
\overset{(c)}{=} &\Pr[\neg\mathsf{Bad}] \cdot \frac{1}{s} \cdot \Pr[b = 1 : b \leftarrow \mathbf{G_3} | \neg\mathsf{Bad}] \\
&+ \Pr[\neg\mathsf{Bad}] \cdot \frac{s-1}{s} \cdot \Pr[b = 1 : b \leftarrow \mathbf{G_4}, l \neq i | \neg\mathsf{Bad}] \\
\geq &\Pr[\neg\mathsf{Bad}] \cdot \frac{1}{s} \cdot \Pr[b = 1 : b \leftarrow \mathbf{G_3} | \neg\mathsf{Bad}] \overset{(d)}{=} \frac{1}{s} \cdot \Pr[b = 1 : b \leftarrow \mathbf{G_3}] .
\end{aligned}$$

Here (a) and (d) follow from the fact that outputting 1 by both game $\mathbf{G_3}$ and $\mathbf{G_4}$ implies that Bad does not occur. The parameter i introduces in (b) satisfies $\mathsf{Reject}_{key}(pk_i, sk_i) = 1$ and $\mathsf{Reject}_{key}(pk_j, sk_j) = 0$ for any $j < i$. Notice that such i must exist under the condition that Bad does not occur. (c) follows from the fact that, when $l = i$, the computation process of game $\mathbf{G_3}$ and $\mathbf{G_4}$ are actually identical according to their definitions. Now we have

$$\Pr[1 \leftarrow \mathbf{G_3}] \leq s \cdot \Pr[1 \leftarrow \mathbf{G_4}] . \tag{23}$$

Game $\mathbf{G_5}$: In this game, the $\mathsf{TrimGen}_3$ is replaced by $\mathsf{TrimGen}_4$, which is identical to $\mathsf{TrimGen}_3$ except that it first samples $c \leftarrow_{\$} \{0, 1\}$ and then replaces the flag event Bad by $c = 0$. That is, if $c = 0$, game $\mathbf{G_5}$ aborts and outputs 0. Otherwise, $\mathsf{TrimGen}_4$ takes (pk_l, sk_l) where $l \leftarrow_{\$} [s]$ as the final return.

It can be noted that game $\mathbf{G_5}$ no longer checks whether Bad occurs; instead, it guesses whether Bad occurs via a random coin toss, i.e., the sampling of the single bit c. Now, we can compute

$$\begin{aligned}
\Pr[b = 1 : b \leftarrow \mathbf{G_4}] &\overset{(a)}{=} \Pr[b = 1 \wedge \neg\mathsf{Bad} : b \leftarrow \mathbf{G_4}] \\
&\overset{(b)}{=} 2 \cdot \Pr[b = 1 \wedge \neg\mathsf{Bad} \wedge c = 1 : b \leftarrow \mathbf{G_4}, c \leftarrow_{\$} \{0, 1\}] \\
&\overset{(c)}{=} 2 \cdot \Pr[b = 1 \wedge \neg\mathsf{Bad} \wedge c = 1 : b \leftarrow \mathbf{G_5}] \\
&\leq 2 \cdot \Pr[b = 1 \wedge c = 1 : b \leftarrow \mathbf{G_5}] \overset{(d)}{=} 2 \cdot \Pr[b = 1 : b \leftarrow \mathbf{G_5}] .
\end{aligned}$$

Here (a) follows from the fact that outputting 1 by game $\mathbf{G_4}$ implies that Bad does not occur, and (d) follows from the fact that outputting 1 by game $\mathbf{G_5}$ implies that $c = 1$. (b) uses the fact game $\mathbf{G_4}$ is independent with c. (c) is because the computation process of game $\mathbf{G_4}$ and $\mathbf{G_5}$ are identical under the condition that $\neg\mathsf{Bad} \wedge c = 1$. Now we have

$$\Pr[1 \leftarrow \mathbf{G_4}] \leq 2 \cdot \Pr[1 \leftarrow \mathbf{G_5}] . \tag{24}$$

Game $\mathbf{G_6}$: In this game, the $\mathsf{TrimGen_4}$ is replaced by KeyGen, and the adversary $\mathcal{A}$ is replaced by the following adversary $\mathcal{A}_1$:

– $\mathcal{A}_1$ samples $c \leftarrow_\$ \{0,1\}$, runs $\mathcal{A}$ and simulates the random oracle H for $\mathcal{A}$. If $c = 0$, $\mathcal{A}_1$ outputs $b' = \perp$. Otherwise, $\mathcal{A}_1$ outputs the final output of $\mathcal{A}$.

In game $\mathbf{G_6}$, the judgment of c made by $\mathsf{TrimGen_4}$ is transferred to $\mathcal{A}_1$, and game $\mathbf{G_6}$ no longer requires rejection sampling when generating (pk, sk). In fact, it is not hard to see that game $\mathbf{G_6}$ is an equivalent rewrite of game $\mathbf{G_5}$, and thus

$$\Pr[1 \leftarrow \mathbf{G_5}] = \Pr[1 \leftarrow \mathbf{G_6}]. \tag{25}$$

Now by combining Eq. (19) to Eq. (25), we obtain

$$\mathrm{Adv}^{\mathsf{IND\text{-}CPA}}_{\mathsf{TrimPKE},\mathcal{A}} \leq 2s \cdot \Pr[1 \leftarrow \mathbf{G_6}] + (r_{\mathrm{key}})^s + \frac{2q_{\mathsf{E}}}{\sqrt{|\mathcal{R}|}}. \tag{26}$$

Next, by following a similar proof idea of the game sequence $\mathbf{G_2}$ to $\mathbf{G_6}$, we introduce the game sequence $\mathbf{G_7}$ to $\mathbf{G_{11}}$ as shown in Figure 12 to remove the rejection sampling of the $\mathsf{TrimEnc_1}$ used by game $\mathbf{G_6}$.

Game $\mathbf{G_7}$: This game is the same as game $\mathbf{G_6}$, except that the encryption algorithm $\mathsf{TrimEnc_1}$ is replaced by $\mathsf{TrimEnc_2}$. Compared with the $\mathsf{TrimEnc_1}$ defined in Figure 7, $\mathsf{TrimEnc_2}$ sets the repetition upper bound of the while loop as a positive integer t, i.e., if the (r, c) generated by the t-th repetition still results in $\mathsf{Reject}_{enc}(pk, m, r, c) = 0$, game $\mathbf{G_7}$ aborts and outputs 0.

Similar to the proof of the game hop $\mathbf{G_1}$ to $\mathbf{G_2}$, by the definition of r_{enc} given in Eq. (17), we have

$$|\Pr[1 \leftarrow \mathbf{G_6}] - \Pr[1 \leftarrow \mathbf{G_7}]| \leq (r_{\mathrm{enc}})^t. \tag{27}$$

Game $\mathbf{G_8}$: In this game, the $\mathsf{TrimEnc_2}$ is replaced by $\mathsf{TrimEnc_3}$, which has the following two differences from $\mathsf{TrimEnc_2}$:

1. $\mathsf{TrimEnc_3}$ no longer executes a while loop, but instead executes a for loop to generate $(r_1, c_1), \ldots, (r_t, c_t)$.
2. Let Bad' be the event that $(r_1, c_1), \ldots, (r_t, c_t)$ satisfy $\mathsf{Reject}_{enc}(pk, m, r, c_1) = \ldots = \mathsf{Reject}_{enc}(pk, m, r, c_t) = 0$. Then, if Bad' occurs, game $\mathbf{G_8}$ aborts and outputs 0. Otherwise, there must exist $1 \leq i \leq t$ s.t. $\mathsf{Reject}_{enc}(pk, m, r, c_i) = 1$ and $\mathsf{Reject}_{enc}(pk, m, r, c_j) = 0$ for any $j < i$, and then $\mathsf{TrimEnc_3}$ takes this c_i as the final return.

Obviously, game $\mathbf{G_8}$ is an equivalent rewrite of game $\mathbf{G_7}$, and thus

$$\Pr[1 \leftarrow \mathbf{G_7}] = \Pr[1 \leftarrow \mathbf{G_8}]. \tag{28}$$

Game $\mathbf{G_9}$: In this game, the $\mathsf{TrimEnc_3}$ is replaced by $\mathsf{TrimEnc_4}$, which is identical to $\mathsf{TrimEnc_3}$ except that if Bad' does not occur, it takes c_l where $l \leftarrow_\$ [t]$ as the final return.

GAMES $\mathbf{G_7}$-$\mathbf{G_{11}}$

$1: (pk, sk) \leftarrow \mathsf{KeyGen}$ $\quad //\mathbf{G_7}\text{-}\mathbf{G_{11}}$
$2: (m_0^*, m_1^*) \leftarrow \mathcal{A}_1(pk)$ $\quad //\mathbf{G_7}\text{-}\mathbf{G_{10}}$
$\quad (m_0^*, m_1^*) \leftarrow \mathcal{B}(pk)$ $\quad //\mathbf{G_{11}}$
$3: b \leftarrow_\$ \{0,1\}$ $\quad //\mathbf{G_7}\text{-}\mathbf{G_{11}}$
$\quad c^* \leftarrow \mathsf{TrimEnc}_{\mathbf{i}-5}(pk, m_b^*)$ $\quad //\mathbf{G}_{\mathbf{7}\le \mathbf{i} \le \mathbf{10}}$
$\quad c^* \leftarrow \mathsf{Encrypt}(pk, m_b^*)$ $\quad //\mathbf{G_{11}}$
$4: b' \leftarrow \mathcal{A}_1(pk, c^*)$ $\quad //\mathbf{G_7}\text{-}\mathbf{G_{10}}$
$\quad b' \leftarrow \mathcal{B}(pk, c^*)$ $\quad //\mathbf{G_{11}}$
$5:$ **return** $[\![b = b']\!]$ $\quad //\mathbf{G_7}\text{-}\mathbf{G_{11}}$

$\mathsf{TrimEnc}_3(pk, m)$

$1:$ **for** k **from** 1 **to** t
$\quad r_k \leftarrow_\$ \mathcal{R}$
$\quad c_k \leftarrow \mathsf{Encrypt}(pk, m, r_k)$
end for
$2:$ **if** Bad' **occurs, abort**
$\quad$ **game return 0**
else return c_i

$\mathsf{TrimEnc}_4(pk, m)$

$1:$ **for** k **from** 1 **to** t
$\quad r_k \leftarrow_\$ \mathcal{R}$
$\quad c_k \leftarrow \mathsf{Encrypt}(pk, m, r_k)$
end for
$2:$ **if** Bad' **occurs, abort**
$\quad$ **game return 0**
else
$\quad l \leftarrow_\$ [t]$
$\quad$ **return** c_l

$\mathsf{TrimEnc}_2(pk, m)$

$1: n = 1, r \leftarrow_\$ \mathcal{R}$
$\quad c \leftarrow \mathsf{Encrypt}(pk, m, r)$
$2:$ **while** $\mathsf{Reject}_{enc}(pk, m, r, c) = 0 \wedge n < t$
$\quad n = n + 1$
$\quad r \leftarrow_\$ \mathcal{R}$
$\quad c \leftarrow \mathsf{Encrypt}(pk, m, r)$
end while
$3:$ **if** $\mathsf{Reject}_{enc}(pk, m, r, c) = 0$**, abort**
$\quad$ **game return 0**
else return c

$\mathsf{TrimEnc}_5(pk, m)$

$1: d \leftarrow_\$ \{0,1\}$
$2:$ **for** k **from** 1 **to** t
$\quad r_k \leftarrow_\$ \mathcal{R}$
$\quad c_k \leftarrow \mathsf{Encrypt}(pk, m, r_k)$
end for
$3:$ **if** $d = 0$ **occurs, abort**
$\quad$ **game return 0**
else
$\quad l \leftarrow_\$ [t]$
$\quad$ **return** c_l

Fig. 12: Games $\mathbf{G_7}$ to $\mathbf{G_{11}}$ in the proof of Theorem 2.

Similar to the proof of the game hop $\mathbf{G_3}$ to $\mathbf{G_4}$, we can compute

$$\Pr[1 \leftarrow \mathbf{G_8}] \le t \cdot \Pr[1 \leftarrow \mathbf{G_9}]. \tag{29}$$

Game $\mathbf{G_{10}}$: In this game, the $\mathsf{TrimEnc}_4$ is replaced by $\mathsf{TrimEnc}_5$, which is identical to $\mathsf{TrimEnc}_4$ except that if first samples $d \leftarrow_\$ \{0,1\}$ and then replaces the flag event Bad' by $d = 0$. That is, if $d = 0$, game $\mathbf{G_{10}}$ aborts and outputs 0. Otherwise, $\mathsf{TrimEnc}_4$ takes c_l where $l \leftarrow_\$ [t]$ as the final return.

Similar to the proof of the game hop $\mathbf{G_4}$ to $\mathbf{G_5}$, we can compute

$$\Pr[1 \leftarrow \mathbf{G_9}] \le 2 \cdot \Pr[1 \leftarrow \mathbf{G_{10}}]. \tag{30}$$

Game $\mathbf{G_{11}}$: In this game, the $\mathsf{TrimEnc}_5$ is replaced by $\mathsf{Encrypt}$, and the adversary $\mathcal{A}_1$ is replaced by the following adversary $\mathcal{B}$:

- $\mathcal{B}$ samples $d \leftarrow_\$ \{0,1\}$ and runs $\mathcal{A}_1$. If $d = 0$, $\mathcal{B}$ outputs $b' = \perp$. Otherwise, $\mathcal{B}$ outputs the final output of $\mathcal{A}_1$.

Obviously we have $\mathcal{T}_\mathcal{B} \approx \mathcal{T}_{\mathcal{A}_1} (\approx \mathcal{T}_\mathcal{A})$, and game $\mathbf{G_{11}}$ is an equivalent rewrite of game $\mathbf{G_{10}}$. In addition, game $\mathbf{G_{11}}$ can be also viewed as an $\mathsf{IND\text{-}CPA}$ game of PKE with the adversary $\mathcal{B}$, and thus

$$\Pr[1 \leftarrow \mathbf{G_{10}}] = \Pr[1 \leftarrow \mathbf{G_{11}}] = \mathrm{Adv}^{\mathsf{IND\text{-}CPA}}_{\mathsf{PKE},\mathcal{B}}. \tag{31}$$

Now by combining Eq. (26) to Eq. (31), we can obtain

$$\mathrm{Adv}^{\mathsf{IND\text{-}CPA}}_{\mathsf{TrimPKE},\mathcal{A}} \leq 4st \cdot \mathrm{Adv}^{\mathsf{IND\text{-}CPA}}_{\mathsf{PKE},\mathcal{B}} + 2s \cdot (r_{enc})^t + (r_{key})^s + \frac{2q_{\mathsf{E}}}{\sqrt{|\mathcal{R}|}}.$$

This completes the proof of Theorem 2. □

6.2 IND-CCA Security Proof of TrimKEM in the QROM

Notably, the KEM scheme TrimKEM shown in Figure 10 is obtained by applying the FO transformation to the public-key encryption scheme TrimPKE. Hence, by combining the security proofs of FO given in [31,35,47] with the above Theorem 2, we have the following security theorems for TrimKEM.

Theorem 3. *Let $\mathcal{A}$ be an* IND-CCA *adversary against* TrimKEM*, making q_{D} classical queries to the decapsulation oracle, and making q_{E}, q_{F}, q_{G} and q_{H} quantum queries to the random oracle* E*,* F*,* G *and* H*, respectively. Then we can construct an* IND-CPA *adversary $\mathcal{B}$ against* PKE *such that $\mathcal{T}_{\mathcal{B}} \approx \mathcal{T}_{\mathcal{A}}$ and*

$$\begin{aligned}\mathrm{Adv}^{\mathsf{IND\text{-}CCA}}_{\mathsf{TrimKEM},\mathcal{A}} \leq & 2(q_{\mathsf{G}} + q_{\mathsf{F}})\sqrt{4st \cdot \mathrm{Adv}^{\mathsf{IND\text{-}CPA}}_{\mathsf{PKE},\mathcal{B}} + 2s \cdot (r_{enc})^t + (r_{key})^s + \frac{2q_{\mathsf{E}}}{\sqrt{|\mathcal{R}|}}} \\ & + \frac{11q_{\mathsf{F}} + 2q_{\mathsf{G}} + 2q_{\mathsf{H}}}{\sqrt{|\mathcal{M}|}} + 4q_{\mathsf{G}} \cdot \sqrt{\delta_{\mathsf{TrimPKE}}}\end{aligned}$$

holds for any positive integers s and t. Here r_{key} and r_{enc} are defined in Eq. (17), $\mathcal{M}/\mathcal{R}$ is the message/randomness space of PKE*, and $\delta_{\mathsf{TrimPKE}}$ is the worst-case DFR of* TrimPKE*.*

Theorem 3 demonstrates that, after applying our Trim-protection to PKE to obtain TrimPKE, we obtain an IND-CCA-secure KEM in the QROM via the FO transformation, with explicit dependence on the rejection probabilities and on the worst-case DFR of TrimPKE. The bound holds for any positive integers s and t, so one can balance the reduction terms $2s \cdot (r_{\mathrm{enc}})^t$ and $(r_{\mathrm{key}})^s$ against the multiplicative factor $4st$ in the usual way. Thus trimming changes concrete security through explicit support-loss terms and any scheme-specific estimator shift; the former are part of the theorem, while the latter must be reported per instantiation rather than treated as universally negligible.

We point out that the first square-root of the security bound of Theorem 3 can be avoided if we instead combine the tighter IND-CCA security proofs of FO given in [38,26]. In addition, if we apply the transformation $\mathsf{FOAC}_0(= \mathsf{FO} \circ \mathsf{ACWC}_0)$ defined in [18,27] to TrimPKE to obtain the TrimKEM, the worst-case DFR of TrimPKE (i.e. $\delta_{\mathsf{TrimPKE}}$) used in the security bound of Theorem 3 could be replaced by the average-case DFR of TrimPKE.

References

1. Albrecht, M.R., Player, R., Scott, S.: On the concrete hardness of Learning with Errors. Journal of Mathematical Cryptology **9**(3), 169–203 (2015). https://doi.org/doi:10.1515/jmc-2015-0016
2. Ambainis, A., Hamburg, M., Unruh, D.: Quantum security proofs using semi-classical oracles. In: Advances in Cryptology - CRYPTO 2019 - 39th Annual International Cryptology Conference, Santa Barbara, CA, USA, August 18-22, 2019, Proceedings, Part II. pp. 269–295. Springer (2019). https://doi.org/10.1007/978-3-030-26951-7_10
3. Bai, S., Galbraith, S.D.: Lattice Decoding Attacks on Binary LWE. In: Susilo, W., Mu, Y. (eds.) Information Security and Privacy. pp. 322–337. Springer International Publishing, Cham (2014)
4. Bernstein, D.J., Chuengsatiansup, C., Lange, T., van Vredendaal, C.: NTRU Prime: Reducing Attack Surface at Low Cost. In: Adams, C., Camenisch, J. (eds.) Selected Areas in Cryptography – SAC 2017. pp. 235–260. Springer International Publishing, Cham (2018)
5. Biasioli, B., Marcolla, C., Calderini, M., Mono, J.: Improving and Automating BFV Parameters Selection: An Average-Case Approach. Cryptology ePrint Archive, Paper 2023/600 (2023), https://eprint.iacr.org/2023/600
6. Bos, J., Ducas, L., Kiltz, E., Lepoint, T., Lyubashevsky, V., Schanck, J.M., Schwabe, P., Seiler, G., Stehle, D.: CRYSTALS - Kyber: A CCA-Secure Module-Lattice-Based KEM. In: 2018 IEEE European Symposium on Security and Privacy (EuroS&P). pp. 353–367 (2018). https://doi.org/10.1109/EuroSP.2018.00032
7. Bossuat, J.P., Costache, A., Mouchet, C., Nürnberger, L., Troncoso-Pastoriza, J.R.: Accurate and Composable Noise Estimates for CKKS with Application to Exact HE Computation. IACR Communications in Cryptology **2**(2) (2025). https://doi.org/10.62056/a3n5tx4e-
8. Brakerski, Z.: Fully Homomorphic Encryption without Modulus Switching from Classical GapSVP. In: Safavi-Naini, R., Canetti, R. (eds.) Advances in Cryptology – CRYPTO 2012. pp. 868–886. Springer Berlin Heidelberg, Berlin, Heidelberg (2012)
9. Brakerski, Z., Gentry, C., Vaikuntanathan, V.: (Leveled) Fully Homomorphic Encryption without Bootstrapping. ACM Trans. Comput. Theory **6**(3) (Jul 2014). https://doi.org/10.1145/2633600, https://doi.org/10.1145/2633600
10. Checri, M., Sirdey, R., Boudguiga, A., Bultel, J.P.: On the Practical CPAD Security of "exact" and Threshold FHE Schemes and Libraries. In: Advances in Cryptology – CRYPTO 2024: 44th Annual International Cryptology Conference, Santa Barbara, CA, USA, August 18–22, 2024, Proceedings, Part III. p. 3–33. Springer-Verlag, Berlin, Heidelberg (2024). https://doi.org/10.1007/978-3-031-68382-4_1, https://doi.org/10.1007/978-3-031-68382-4_1
11. Cheon, J.H., Choe, H., Passelègue, A., Stehlé, D., Suvanto, E.: Attacks Against the IND-CPAD Security of Exact FHE Schemes. In: Proceedings of the 2024 on ACM SIGSAC Conference on Computer and Communications Security. p. 2505–2519. CCS '24, Association for Computing Machinery, New York, NY, USA (2024). https://doi.org/10.1145/3658644.3690341, https://doi.org/10.1145/3658644.3690341
12. Cheon, J.H., Han, K., Kim, A., Kim, M., Song, Y.: A Full RNS Variant of Approximate Homomorphic Encryption. In: Selected Areas in Cryptography – SAC 2018: 25th International Conference, Calgary, AB, Canada, August 15–17, 2018,

Revised Selected Papers. p. 347–368. Springer-Verlag, Berlin, Heidelberg (2018). https://doi.org/10.1007/978-3-030-10970-7_16, https://doi.org/10.1007/978-3-030-10970-7_16

13. Chung, C.M.M., Hwang, V., Kannwischer, M.J., Seiler, G., Shih, C.J., Yang, B.Y.: NTT Multiplication for NTT-unfriendly Rings: New Speed Records for Saber and NTRU on Cortex-M4 and AVX2. IACR Transactions on Cryptographic Hardware and Embedded Systems **2021**(2), 159–188 (Feb 2021). https://doi.org/10.46586/tches.v2021.i2.159-188
14. Costache, A., Nürnberger, L., Player, R.: Optimisations and Tradeoffs for HElib. In: Rosulek, M. (ed.) Topics in Cryptology – CT-RSA 2023. pp. 29–53. Springer International Publishing, Cham (2023)
15. D Anvers, J.P., Vercauteren, F., Verbauwhede, I., Ding, J., Steinwandt, R.: The Impact of Error Dependencies on Ring/Mod-LWE/LWR Based Schemes. Post-Quantum Cryptography **11505** (2019-07-14)
16. D'Anvers, J.P., Karmakar, A., Sinha Roy, S., Vercauteren, F.: Saber: Module-LWR Based Key Exchange, CPA-Secure Encryption and CCA-Secure KEM. In: Joux, A., Nitaj, A., Rachidi, T. (eds.) Progress in Cryptology – AFRICACRYPT 2018. pp. 282–305. Springer International Publishing, Cham (2018)
17. Ducas, L., van Woerden, W.: NTRU Fatigue: How Stretched is Overstretched? In: Tibouchi, M., Wang, H. (eds.) Advances in Cryptology – ASIACRYPT 2021. pp. 3–32. Springer International Publishing, Cham (2021)
18. Duman, J., Hövelmanns, K., Kiltz, E., Lyubashevsky, V., Seiler, G., Unruh, D.: A thorough treatment of highly-efficient NTRU instantiations. In: Public-Key Cryptography - PKC 2023 - 26th IACR International Conference on Practice and Theory of Public-Key Cryptography, Atlanta, GA, USA, May 7-10, 2023, Proceedings, Part I. pp. 65–94. Springer (2023). https://doi.org/10.1007/978-3-031-31368-4_3
19. D'Anvers, J.P., Guo, Q., Johansson, T., Nilsson, A., Vercauteren, F., Verbauwhede, I.: Decryption Failure Attacks on IND-CCA Secure Lattice-Based Schemes. In: Public-Key Cryptography – PKC 2019: 22nd IACR International Conference on Practice and Theory of Public-Key Cryptography, Beijing, China, April 14-17, 2019, Proceedings, Part II. p. 565–598. Springer-Verlag, Berlin, Heidelberg (2019). https://doi.org/10.1007/978-3-030-17259-6_19, https://doi.org/10.1007/978-3-030-17259-6_19
20. D'Anvers, J.P., Rossi, M., Virdia, F.: (One) Failure Is Not an Option: Bootstrapping the Search for Failures; Lattice-Based Encryption Schemes. In: Advances in Cryptology – EUROCRYPT 2020: 39th Annual International Conference on the Theory and Applications of Cryptographic Techniques, Zagreb, Croatia, May 10–14, 2020, Proceedings, Part III. p. 3–33. Springer-Verlag, Berlin, Heidelberg (2020). https://doi.org/10.1007/978-3-030-45727-3_1, https://doi.org/10.1007/978-3-030-45727-3_1
21. Fabšič, T., Hromada, V., Stankovski, P., Zajac, P., Guo, Q., Johansson, T.: A Reaction Attack on the QC-LDPC McEliece Cryptosystem. In: Lange, T., Takagi, T. (eds.) Post-Quantum Cryptography. pp. 51–68. Springer International Publishing, Cham (2017)
22. Fan, J., Vercauteren, F.: Somewhat Practical Fully Homomorphic Encryption. IACR Cryptol. ePrint Arch. **2012**, 144 (2012), https://api.semanticscholar.org/CorpusID:1467571
23. Fouque, P.A., Hoffstein, J., Kirchner, P., Lyubashevsky, V., Pornin, T., Prest, T., Ricosset, T., Seiler, G., Whyte, W., Zhang, Z.: Falcon: Fast-Fourier Lattice-based Compact Signatures over NTRU (2019)

24. Fouque, P.A., Kirchner, P., Pornin, T., Yu, Y.: BAT: Small and Fast KEM over NTRU Lattices. IACR Transactions on Cryptographic Hardware and Embedded Systems **2022**, 240–265 (02 2022). https://doi.org/10.46586/tches.v2022.i2.240-265
25. Gao, M., Zheng, H.: A Critique on Average-Case Noise Analysis in RLWE-Based Homomorphic Encryption. Cryptology ePrint Archive, Paper 2025/1036 (2025), https://eprint.iacr.org/2025/1036
26. Ge, J., Liao, H., Xue, R.: Measure-rewind-extract: Tighter proofs of one-way to hiding and CCA security in the quantum random oracle model. In: Advances in Cryptology - ASIACRYPT 2024 - 30th International Conference on the Theory and Application of Cryptology and Information Security, Kolkata, India, December 9-13, 2024, Proceedings, Part IV. Lecture Notes in Computer Science, vol. 15487, pp. 3–34. Springer (2024). https://doi.org/10.1007/978-981-96-0894-2_1
27. Ge, J., Yang, K., Yu, Y., Yu, Y.: Fujisaki-okamoto transformation under average-case decryption error: Tighter and more general proofs with applications to PQC. Cryptology ePrint Archive, Paper 2025/1641 (2025), https://eprint.iacr.org/2025/1641
28. Guo, Q., Johansson, T., Stankovski, P.: A key recovery attack on MDPC with CCA security using decoding errors. In: Advances in Cryptology - ASIACRYPT 2016 - 22nd International Conference on the Theory and Application of Cryptology and Information Security, Proceedings. Lecture Notes in Computer Science (including subseries Lecture Notes in Artificial Intelligence and Lecture Notes in Bioinformatics), vol. 10031 LNCS, pp. 789–815. Springer, Germany (2016). https://doi.org/10.1007/978-3-662-53887-6_29, 22nd Annual International Conference on the Theory and Applications of Cryptology and Information Security (ASIACRYPT), 2016 ; Conference date: 04-12-2016 Through 08-12-2016
29. Guo, Q., Johansson, T., Yang, J.: A Novel CCA Attack Using Decryption Errors Against LAC. In: Galbraith, S.D., Moriai, S. (eds.) Advances in Cryptology – ASIACRYPT 2019. pp. 82–111. Springer International Publishing, Cham (2019)
30. Hoffstein, J., Pipher, J., Silverman, J.H.: NTRU: A ring-based public key cryptosystem. In: Buhler, J.P. (ed.) Algorithmic Number Theory. pp. 267–288. Springer Berlin Heidelberg, Berlin, Heidelberg (1998)
31. Hofheinz, D., Hövelmanns, K., Kiltz, E.: A modular analysis of the fujisaki-okamoto transformation. In: Theory of Cryptography Conference. pp. 341–371. Springer (2017). https://doi.org/10.1007/978-3-319-70500-2_12
32. Hong, G.H., Woo, J., Kim, J., Kim, M., Lee, H., Park, J.H.: More NTRU+Sign Signatures from Cyclotomic Trinomials. Cryptology ePrint Archive, Paper 2025/612 (2025), https://eprint.iacr.org/2025/612
33. Howgrave-Graham, N., Nguyen, P.Q., Pointcheval, D., Proos, J., Silverman, J.H., Singer, A., Whyte, W.: The Impact of Decryption Failures on the Security of NTRU Encryption. In: Boneh, D. (ed.) Advances in Cryptology - CRYPTO 2003. pp. 226–246. Springer Berlin Heidelberg, Berlin, Heidelberg (2003)
34. Hülsing, A., Rijneveld, J., Schanck, J., Schwabe, P.: High-Speed Key Encapsulation from NTRU. In: Fischer, W., Homma, N. (eds.) Cryptographic Hardware and Embedded Systems – CHES 2017. pp. 232–252. Springer International Publishing, Cham (2017)
35. Jiang, H., Zhang, Z., Chen, L., Wang, H., Ma, Z.: Ind-cca-secure key encapsulation mechanism in the quantum random oracle model, revisited. In: Advances in Cryptology - CRYPTO 2018 - 38th Annual International Cryptology Conference, Santa Barbara, CA, USA, August 19-23, 2018, Proceedings, Part III. pp. 96–125. Springer (2018). https://doi.org/10.1007/978-3-319-96878-0_4

36. Kannan, R.: Minkowski's Convex Body Theorem and Integer Programming. Math. Oper. Res. **12**(3), 415–440 (Aug 1987)
37. Kim, J., Park, J.H.: NTRU+: Compact Construction of NTRU Using Simple Encoding Method. IEEE Transactions on Information Forensics and Security **18**, 4760–4774 (2023). https://doi.org/10.1109/TIFS.2023.3299172
38. Kuchta, V., Sakzad, A., Stehlé, D., Steinfeld, R., Sun, S.: Measure-rewind-measure: Tighter quantum random oracle model proofs for one-way to hiding and CCA security. In: Advances in Cryptology - EUROCRYPT 2020 - 39th Annual International Conference on the Theory and Applications of Cryptographic Techniques, Zagreb, Croatia, May 10-14, 2020, Proceedings, Part III. pp. 703–728. Springer (2020). https://doi.org/10.1007/978-3-030-45727-3_24
39. Li, B., Micciancio, D.: On the Security of Homomorphic Encryption on Approximate Numbers. In: Canteaut, A., Standaert, F.X. (eds.) Advances in Cryptology – EUROCRYPT 2021. pp. 648–677. Springer International Publishing, Cham (2021)
40. Liang, Z., Fang, B., Zheng, J., Zhao, Y.: Compact and Efficient KEMs over NTRU Lattices (2022), https://arxiv.org/abs/2205.05413
41. Liu, S., Sakzad, A.: Lattice codes for crystals-kyber. Designs, Codes and Cryptography (May 2025). https://doi.org/10.1007/s10623-025-01640-w, https://doi.org/10.1007/s10623-025-01640-w
42. Liu, Y., Zhang, Y., Lu, X., Cheng, Y., Yin, Y.: DAWN: Smaller and Faster NTRU Encryption via Double Encoding. Cryptology ePrint Archive, Paper 2025/1520, to appear in Proceedings of ASIACRYPT 2025. (2025), https://eprint.iacr.org/2025/1520
43. Liu, Y., Lu, X., Zhang, Y., Wang, R., Liu, Z., Wang, K.: Compact Lifting for NTT-Unfriendly Modulus. In: Information Security and Privacy: 30th Australasian Conference, ACISP 2025, Wollongong, NSW, Australia, July 14–16, 2025, Proceedings, Part II. p. 277–296. Springer-Verlag, Berlin, Heidelberg (2025). https://doi.org/10.1007/978-981-96-9098-5_14, https://doi.org/10.1007/978-981-96-9098-5_14
44. Liu, Z., Wang, Y., Fisch, B.: IND-CPA-D of Relaxed Functional Bootstrapping: A New Attack, A General Fix, and A Stronger Model. In: Proceedings of the 2025 ACM SIGSAC Conference on Computer and Communications Security. p. 2907–2921. CCS '25, Association for Computing Machinery, New York, NY, USA (2025). https://doi.org/10.1145/3719027.3765170, https://doi.org/10.1145/3719027.3765170
45. Lu, X., Liu, Y., Zhang, Z., Jia, D., Xue, H., He, J., Li, B., Wang, K.: LAC: Practical Ring-LWE Based Public-Key Encryption with Byte-Level Modulus. Cryptology ePrint Archive, Paper 2018/1009 (2018), https://eprint.iacr.org/2018/1009
46. Lyubashevsky, V., Seiler, G.: NTTRU: Truly Fast NTRU Using NTT. IACR Transactions on Cryptographic Hardware and Embedded Systems **2019**(3), 180–201 (May 2019). https://doi.org/10.13154/tches.v2019.i3.180-201
47. Maram, V., Xagawa, K.: Post-quantum anonymity of kyber. In: Public-Key Cryptography - PKC 2023 - 26th IACR International Conference on Practice and Theory of Public-Key Cryptography, Atlanta, GA, USA, May 7-10, 2023, Proceedings, Part I. Lecture Notes in Computer Science, vol. 13940, pp. 3–35. Springer (2023). https://doi.org/10.1007/978-3-031-31368-4_1
48. Schnorr, C.P.: Block Reduced Lattice Bases and Successive Minima. Combinatorics, Probability and Computing **3**(4), 507–522 (1994). https://doi.org/10.1017/S0963548300001371

49. Schnorr, C.P., Euchner, M.: Lattice basis reduction: Improved practical algorithms and solving subset sum problems. Mathematical Programming **66**(1), 181–199 (Aug 1994). https://doi.org/10.1007/BF01581144, https://doi.org/10.1007/BF01581144
50. Shoup, V.: Sequences of games: a tool for taming complexity in security proofs. IACR Cryptol. ePrint Arch. p. 332 (2004)
51. Zhang, J., Feng, D., Yan, D.: NEV: Faster and Smaller NTRU Encryption Using Vector Decoding. In: Guo, J., Steinfeld, R. (eds.) Advances in Cryptology – ASIACRYPT 2023. pp. 157–189. Springer Nature Singapore, Singapore (2023)

A Algorithms

A.1 TrimLAC

We keep the notation of the trim upper bound as B_{inner}. Let $\Psi_{n,hw}$ denote the distribution of the polynomial whose coefficients contain $hw/2$ 1s and $hw/2$ -1s, and the rest are all 0s. For a polynomial a, we use $\lfloor a \rceil$ to denote the polynomial after the rounding operation such that $\lfloor a \rceil_{[i]} = \lfloor a_{[i]} \rceil$. The functions ECCEnc and ECCDec are the encoding and decoding functions corresponding to the ECC. We use the symmetric primitives as: a pseudorandom function PRF, an extendable-output function XOF, and two variable-length input hash functions G and H. In Algorithms 2 and 3, we do not replicate the full framework of TrimPKE 7; instead, we only adapt the relevant parts. Since the seeds ρ_a and (ρ_s, ρ_e) are decoupled, and rejection sampling does not apply to the sampling of a, the concrete algorithms can derive randomness partially while still following the same deterministic-reseeding pattern. We use these appendix algorithms only as case-study instantiations of the generic framework; the load-bearing reduction remains the generic TrimPKE/TrimKEM proof.

Algorithm 2 TrimLAC.PKE.KeyGen

Output: the public key pk, the private key sk

1: $\rho_a \leftarrow \mathsf{Sample}(\mathcal{U}(\{0,1\}^{256}))$
2: $a \leftarrow \mathsf{Sample}(\mathcal{U}(\mathcal{R}_n/q\mathcal{R}_n), \rho_a)$
3: $\rho_s \leftarrow \mathsf{Sample}(\mathcal{U}(\{0,1\}^{256}))$
4: $s \leftarrow \mathsf{Sample}(\Psi_{n,hw})$
5: **if** $\|(s\bar{s})_{[1:n/2)}\|_\infty \geq B_{\text{inner}}$
6: go back to step 4
7: $\rho_e \leftarrow \mathsf{Sample}(\mathcal{U}(\{0,1\}^{256}))$
8: $e \leftarrow \mathsf{Sample}(\Psi_{n,hw})$
9: **if** $\|(e\bar{e})_{[1:n/2)}\|_\infty \geq B_{\text{inner}}$
10: go back to step 8
11: $b \leftarrow as + e \pmod q$
12: **return** $(pk := (\rho_a, b), sk := s)$

Algorithm 3 TrimLAC.PKE.Encrypt

Input: the public key $pk := (\rho_a, b)$, the message m, the seed ρ_0
Output: the ciphertext c

$a \leftarrow \mathsf{Sample}(\mathcal{U}(\mathcal{R}_n/q\mathcal{R}_n), \rho_a)$
$\tilde{m} \leftarrow \mathsf{ECCEnc}(m)$
$\rho \leftarrow \rho_0$
$N_\rho \leftarrow 0$
repeat
  $\rho \leftarrow \mathsf{H}(N_\rho \| \rho \| \rho_0)$
  $N_\rho \leftarrow N_\rho + 1$
  $N_{\mathrm{PRF}} \leftarrow 0$
  $r \leftarrow \mathsf{Sample}(\Psi_{n,hw}, \mathsf{PRF}(\rho, N_{\mathrm{PRF}}))$
  $N_{\mathrm{PRF}} \leftarrow N_{\mathrm{PRF}} + 1$
  $e_1 \leftarrow \mathsf{Sample}(\Psi_{n,hw}, \mathsf{PRF}(\rho, N_{\mathrm{PRF}}))$
  $N_{\mathrm{PRF}} \leftarrow N_{\mathrm{PRF}} + 1$
  $e_2 \leftarrow \mathsf{Sample}(\Psi_{n,hw}, \mathsf{PRF}(\rho, N_{\mathrm{PRF}}))$
until $\|(r\bar{r})_{[1:n/2)}\|_\infty < B_{\mathrm{inner}}$ and $\|(e_1\bar{e}_1)_{[1:n/2)}\|_\infty < B_{\mathrm{inner}}$
$c_1 \leftarrow ar + e_1 \pmod q$
$c_2 \leftarrow br + e_2 + \lfloor q \rceil \cdot \tilde{m} \pmod q$
return $c := (c_1, c_2)$

Algorithm 4 TrimLAC.PKE.Decrypt

Input: the private key $sk := s$, the ciphertext $c := (c_1, c_2)$
Output: the message m

$\tilde{m} \leftarrow \lfloor \frac{2}{q} \cdot (c_2 - c_1 s \pmod q) \rceil \pmod 2$
$m \leftarrow \mathsf{ECCDec}(\tilde{m})$
return m

Algorithm 5 TrimLAC.KEM.KeyGen

Output: the public key pk, the private key sk

$(pk, sk') \leftarrow \mathsf{TrimLAC.PKE.KeyGen}$
$\bar{K}' \leftarrow \mathsf{Sample}(\mathbb{Z}_2^{256})$
$sk \leftarrow (sk', pk, \mathsf{H}(pk), \bar{K}')$
return (pk, sk)

Algorithm 6 TrimLAC.KEM.Encapsulate

Input: the public key pk
Output: the ciphertext c, the shared key K

$m \leftarrow \mathsf{Sample}(\mathbb{Z}_2^{256})$
$(\bar{K}, \rho_0) \leftarrow \mathsf{G}(m \| \mathsf{H}(pk))$
$c \leftarrow \mathsf{TrimLAC.PKE.Encrypt}(pk, m, \rho_0)$
$K \leftarrow \mathsf{KDF}(\bar{K} \| \mathsf{H}(c))$
return c, K

Algorithm 7 TrimLAC.KEM.Decapsulate

Input: the private key sk, the ciphertext c
Output: the shared key K

$(sk', pk, \mathsf{H}(pk), \bar{K}') \leftarrow sk$
$m' \leftarrow \mathsf{TrimLAC.PKE.Decrypt}(sk', c)$
$(\bar{K}, \rho_0) \leftarrow \mathsf{G}(m' \| \mathsf{H}(pk))$
if $c \neq \mathsf{TrimLAC.PKE.Encrypt}(pk, m', \rho_0)$
    $K \leftarrow \mathsf{KDF}(\bar{K}' \| \mathsf{H}(c))$
else
    $K \leftarrow \mathsf{KDF}(\bar{K} \| \mathsf{H}(c))$
return K

A.2 TrimDAWN

We set the trim parameters as $\omega_f, \omega_g, \omega_s, \omega_e, \omega_{e_{d_c}}$. Let θ_f, θ_g, θ_s, θ_e, $\theta_{e_{d_c}}$ denote the corresponding calibration tuples used by IsDistorted. For reference, the underlying coefficient samplers have standard deviations σ_f, σ_g, σ_s, σ_e, $\sigma_{e_{d_c}}$ for the distributions $\mathcal{ST}_{n,k_g}, \mathcal{ST}_{n,k_f}$, $\mathcal{ST}_{n,k_s}$, $\mathcal{ST}_{n,k_e}$, $\mathcal{X}_{e_{d_c}}$, where $\mathcal{ST}_{n,k}$ corresponds to the sparse ternary distribution $\{0 : 1-\frac{2k}{n}, 1 : \frac{k}{n}, -1 : \frac{k}{n}\}$ and $\mathcal{X}_{e_{d_c}}$ corresponds to the distribution of the noise polynomial $\boldsymbol{e_{d_c}}$ introduced by ciphertext compression. We use $\pmod{\langle \varphi, \mathbb{Z}_q \rangle}$ to denote modular arithmetic in the quotient ring $\mathbb{Z}_q[x]/\varphi$. Throughout this subsection, we follow the notation of DAWN [42] whenever possible for consistency.

Algorithm 8 SimpleDecoding

Input: three polynomials $\boldsymbol{c}', \boldsymbol{m}', \boldsymbol{f_2} \in \mathcal{R}_{x^{n/2}+1}$
Output: the message polynomial $\boldsymbol{m}$

```
e' ← (c') (mod ⟨x^{n/4} + 1, 2⟩)
c'' ← ((e')^{-1} c') (mod ⟨x^n + 1⟩)
i ← 0
base ← q
for j from 0 to 3
    v ← min(|c''_{[jn/4]} + q|, |c''_{[jn/4]} − q|)
    if v < base
        base ← v
        i ← jn/4
m ← (m' + x^i f_2 e') (mod ⟨x^{n/2} + 1, 2⟩)
if e' = 0
    m ← (m') (mod ⟨x^{n/4} + 1, 2⟩)
else
    m ← (m) (mod ⟨x^{n/4} + 1, 2⟩)
return m
```

Algorithm 9 TrimDAWN.PKE.KeyGen

Output: the public key pk, the private key sk

$g \leftarrow \mathsf{Sample}(\mathcal{ST}^n_{n,k_g})$
$f \leftarrow \mathsf{Sample}(\mathcal{ST}^n_{n,k_f})$
if f or g is not invertible in $\mathcal{R}_n/q\mathcal{R}_n$
    restart key generation
if $\mathsf{IsDistorted}(g, \theta_g, \omega_g) + \mathsf{IsDistorted}(f, \theta_f, \omega_f) \neq 0$
    restart key generation
$f_2 \leftarrow f^{-1} \pmod{\langle x^{n/2}+1, 2\rangle}$
if f_2 is not invertible in $\mathcal{R}_{n/2}/2\mathcal{R}_{n/2}$
    restart key generation
$h \leftarrow (gf^{-1}) \pmod{\langle x^n+1, q\rangle}$
$pk \leftarrow h$
$sk \leftarrow (f, f_2)$
return (pk, sk)

Algorithm 10 TrimDAWN.PKE.Encrypt

Input: the public key pk, the message m, the seed ρ_0
Output: the ciphertext c

$h \leftarrow pk$
$\rho \leftarrow \rho_0$
$N_\rho \leftarrow 0$
repeat
    $\rho \leftarrow \mathsf{H}(N_\rho \| \rho \| \rho_0)$
    $N_\rho \leftarrow N_\rho + 1$
    $N_{\mathrm{PRF}} \leftarrow 0$
    $s \leftarrow \mathsf{Sample}(\mathcal{ST}^n_{n,k_s}, \mathsf{PRF}(\rho, N_{\mathrm{PRF}}))$
    $N_{\mathrm{PRF}} \leftarrow N_{\mathrm{PRF}} + 1$
    $e \leftarrow \mathsf{Sample}(\mathcal{ST}^n_{n,k_e}, \mathsf{PRF}(\rho, N_{\mathrm{PRF}}))$
    $N_{\mathrm{PRF}} \leftarrow N_{\mathrm{PRF}} + 1$
    $c' \leftarrow \left(hs + e + (x^{n/4}+1)^{-1}m\right) \pmod{\langle x^n+1, q\rangle}$
    $c \leftarrow \lfloor \frac{1}{d_c} \cdot c' \rceil$
    $e_{d_c} \leftarrow c' - d_c \cdot c$
    if $d_c \equiv 0 \bmod 2$
        $e_{d_c} \leftarrow 2 \cdot e_{d_c} + \sum_{i=0}^{n-1} x^i$
until $\mathsf{IsDistorted}(s, \theta_s, \omega_s) = 0$ and $\mathsf{IsDistorted}(e, \theta_e, \omega_e) = 0$ and $\mathsf{IsDistorted}(e_{d_c}, \theta_{e_{d_c}}, \omega_{e_{d_c}}) = 0$
return c

Algorithm 11 TrimDAWN.PKE.Decrypt

Input: the ciphertext c, the private key sk
Output: the message m

$m \leftarrow \mathsf{DAWN.PKE.Decrypt}(c, sk)$
return m

Algorithm 12 TrimDAWN.KEM.KeyGen

Output: the public key pk, the private key sk

$(pk, sk') \leftarrow \mathsf{TrimDAWN.PKE.KeyGen}$
$\mathrm{K}' \leftarrow \mathsf{Sample}(\mathcal{U}(\mathcal{M}))$
$sk \leftarrow (\mathrm{K}', sk', pk, \mathsf{H}(pk))$
return (pk, sk)

Algorithm 13 TrimDAWN.KEM.Encapsulation

Input: the public key pk
Output: the ciphertext c, the shared key K

$m \leftarrow \mathsf{Sample}(\mathcal{M})$
$(\bar{\mathrm{K}}, \rho_0) \leftarrow \mathsf{G}(m \| \mathsf{H}(pk))$
$c \leftarrow \mathsf{TrimDAWN.PKE.Encrypt}(pk, m, \rho_0)$
$\mathrm{K} \leftarrow \mathsf{KDF}(\bar{\mathrm{K}}, \mathsf{H}(c))$
return (c, K)

Algorithm 14 TrimDAWN.KEM.Decapsulation

Input: the ciphertext c, the private key sk
Output: the shared key K

$(\mathrm{K}', sk', pk, \mathsf{H}(pk)) \leftarrow sk$
$m \leftarrow \mathsf{TrimDAWN.PKE.Decrypt}(c, sk')$
$(\bar{\mathrm{K}}, \rho_0) \leftarrow \mathsf{G}(m \| \mathsf{H}(pk))$
if $c \neq \mathsf{TrimDAWN.PKE.Encrypt}(pk, m, \rho_0)$
    $K \leftarrow \mathsf{KDF}(\bar{\mathrm{K}}', \mathsf{H}(c))$
else
    $K \leftarrow \mathsf{KDF}(\bar{\mathrm{K}}, \mathsf{H}(c))$
return K